

OpenDDA User's Guide

**Release 300
DD11-300**

OpenDDA

**OpenDDA
User's Guide**

**Release 300
DD11-300
12/98**

Notices and Trademarks

© Copyright 1998 by Honeywell Inc.

Revision 01 – December 18, 1998

While this information is presented in good faith and believed to be accurate, Honeywell disclaims the implied warranties of merchantability and fitness for a particular purpose and makes no express warranties except as may be stated in its written agreement with and for its customer.

In no event is Honeywell liable to anyone for any indirect, special or consequential damages. The information and specifications in this document are subject to change without notice.

Honeywell, **TotalPlant**, and Universal Station^x are U.S. registered trademarks of Honeywell Inc.

Other brand or product names are trademarks of their respective owners.

Honeywell
Industrial Automation and Control
Automation College
2820 West Kelton Lane
Phoenix, AZ 85053-3028
1-800-852-3211

Table of Contents

1. USING THIS GUIDE	17
1.1 Overview	17
Introduction	17
What's in this section	17
Tutorial	17
1.2 About This Document	18
Purpose	18
Audience	18
Software release	18
Organization of this guide	18
Tutorial	18
OpenDDA application reference information	18
Troubleshooting OpenDDA	18
1.3 Conventions Used in This Guide	19
Typed/Displayed text	19
Upper/lowercase letters	19
Optional parameters indicators	19
Applications	19
1.4 Overview of the Sections in This Guide	20
Document sections and descriptions	20
2. INTRODUCTION TO OPENDDA	21
2.1 Overview	21
Introduction	21
DDA Host Platform support	21
OpenDDA Applications	21
OpenDDA's Builder functions	21
OpenDDA's Executive functions	22
How OpenDDA operates	22
OpenDDA requires you know	22
2.2 What Can OpenDDA Do For You?	23
Major features of OpenDDA	23
2.3 Introduction to OpenDDA Applications	24
What is an OpenDDA application?	24
Offline/Online State	24
2.4 OpenDDA Offline Applications	25
Offline directory	25
Offline applications	25
2.5 OpenDDA Online Applications	26
Online directory	26
Online applications	26
2.6 Data Access	27
What is data access?	27
DCS data	27
2.7 Introduction to OpenDDA Application Types	28
OpenDDA application scope	28
High Level Languages (HLL)	28

Table of Contents

2.8	Application Source and External Data	29
	Application source files.....	29
	External data	29
3.	PREPARING TO USE OPENDDA.....	31
3.1	Overview	31
	Introduction.....	31
	Tutorial	31
3.2	OpenDDA Directory Structure	32
	Directory tree.....	32
	Directory contents	33
3.3	The OpenDDA Command	34
	Preparing for the OpenDDA command.....	34
	To enable the OpenDDA command	34
	csh command	34
3.4	Example Files	35
	OpenDDA examples provided	35
	Template starter files.....	35
	Example OpenDDA applications	35
	“Make” and script files	36
3.5	Shared Libraries	38
	Description	38
	Advantages	38
	Location.....	38
	Impact on users.....	38
3.6	Compiler Options	39
	Recommended compiler options for FORTRAN	39
	Suggested compiler option for FORTRAN	39
	Track compiler errors	39
	Recommended compiler options for C	40
	Recommended compiler options for C++.....	40
3.7	Environment Variables	41
	OpenDDA environment variables	41
	DDAOPTS	41
	DDA_REDIRECT_ OUTPUT	41
3.8	A*M Security	42
	Security Switch.....	42
	READONLY.....	42
	RW_LCN_I	42
	READWRIT	42
4.	DEVELOPING OPENDDA APPLICATIONS	45
4.1	Overview	45
	Introduction.....	45
	Tutorial	45
4.2	How to Create a New OpenDDA Application.....	46
	Creating a new OpenDDA application.....	46
	Figure 4-1 Create a New OpenDDA Application	46
	How to create an OpenDDA application.....	47

Table of Contents

4.3	How to Build an OpenDDA Application.....	48
	Building an OpenDDA application	48
	Figure 4-2 How to Build an OpenDDA Application	48
	How to build an OpenDDA application	49
4.4	How to Build When the EDB is Separate.....	51
	Building an application with separate EDB	51
	How to build an application with separate EDB	52
4.5	Developing an OpenDDA Application on a Remote Node.....	54
	Creating an OpenDDA Application	54
4.6	Moving an OpenDDA Application Developed on a Remote Node to an A*M.....	55
	Moving an OpenDDA application.....	55
4.7	How to Install an OpenDDA Application Online	57
	Installing an OpenDDA application online	57
	How to install an application online.....	58
	Installed application execution behavior	58
4.8	How to Execute an OpenDDA Application	59
	Introduction	59
	Executing an OpenDDA offline application from an A*M	59
	How to execute an offline OpenDDA application on an A*M.....	60
	Executing an OpenDDA offline application from a Remote Node	61
	How to execute an offline OpenDDA application on a remote node.....	62
	Executing an OpenDDA online application from an A*M.....	62
	How to execute an online OpenDDA application on an A*M.....	63
5.	OPENDDA APPLICATION DEVELOPMENT FUNCTIONS	65
5.1	Overview.....	65
	Introduction	65
	In this section.....	65
5.2	Using OpenDDA Functions from the Command Line	66
	OpenDDA command line interface	66
	OpenDDA command line components	66
	Composing an OpenDDA command line statement.....	66
	The DDAOPTS environment variable	67
	Using DDAOPTS	67
	How OpenDDA builds the dda command line	68
5.3	Specifying OpenDDA Command Line Functions	69
	Specifying OpenDDA functions	69
	OpenDDA command line qualifiers.....	69
	Filespec arguments	69
	OpenDDA command line options	70
5.4	OpenDDA Command Line Messages	71
	Viewing OpenDDA command line messages	71
	Redirecting Standard Output	72
	Understanding OpenDDA command line error messages.....	72
	Example of OpenDDA command line messages.....	73
	Redirecting command line messages to a file	73
5.5	OpenDDA Completion Status	74
	OpenDDA completion status	74
	OpenDDA completion status values	74
	Using the OpenDDA completion status	74

Table of Contents

5.6	OpenDDA Command Line Help	75
	Obtaining Help from the command line interface	75
5.7	Using OpenDDA Functions from Script Files	76
	Using host platform script files	76
	Script file example	76
5.8	Using OpenDDA Functions in a Program.....	77
	Introduction.....	77
	The SYSTEM call	77
	Examples.....	77
5.9	Parse External Data Block.....	78
	Purpose	78
	Command line qualifier.....	78
	Input file	78
	Output file	78
	Valid options.....	79
	Remote node platform restrictions	79
	Examples.....	80
5.10	Expand EXEC DDA Statements	81
	Purpose	81
	Command line qualifier.....	81
	Input file.....	81
	Output file	82
	Valid options.....	82
	Examples.....	83
5.11	Complete Build.....	84
	Purpose	84
	Command line qualifier.....	84
	Input file	84
	Output file	84
	Valid options.....	85
	Example	85
5.12	Validate Definition File Syntax.....	86
	Purpose	86
	Command line qualifier.....	86
	Input file.....	86
	Output file	86
	Valid options.....	86
	Example	86
	Directives that are no longer required	87
	DEF file optional.....	87
5.13	Install Application	88
	Application Directory	88
	Purpose of Install.....	88
	File protection.....	88
	Command Line Qualifier.....	88
	Input file.....	88
	Output file	88
	Valid options.....	88
	Example	89
	After installation.....	89
	Completion status.....	91

Table of Contents

5.14 De-Install Application	92
Purpose	92
Command line qualifier	92
Input file	92
Output file	92
Valid options	92
Example	92
6. FILE MANAGEMENT	93
6.1 Overview	93
Introduction	93
6.2 OpenDDA Files	94
OpenDDA file types	94
7. USING OPENDDA STATUSES	97
7.1 Overview	97
Introduction	97
OpenDDA Status Value defined	97
OpenDDA Status Mnemonic defined	97
OpenDDA Status Variables	98
7.2 Interpreting OpenDDA Status Values	99
Introduction	99
Status Components	99
Facility Code	99
Message Number	100
Severity	100
Extracting the status components in a C application	101
Example	101
Extracting the status components in a FORTRAN application	102
Example	103
7.3 Determining Corrective Action	104
Introduction	104
Using the Severity component	104
Obtaining OpenDDA Error Mnemonic	104
Example	105
Corrective action	105
8. REMAPPING CIO FILES	107
8.1 Overview	107
Introduction	107
What is a CIO file?	107
Dynamically changing external data	108
8.2 Remapping CIO Functionality	109
The remapping concept	109
The remapping tools	109
Maximum number of CIOs	109
Similarity to Equipment List	110
The EDBs must be structurally identical	110
The CIO signature	110
How EDBs can differ	111
How EDBs cannot differ	111

Table of Contents

9. DYNAMIC SELECTION SETS.....	113
9.1 Overview	113
What will be covered	113
Introduction.....	113
Comparison with Remap CIO	113
The tools for Dynamic Selection Sets	114
Location Specifications that are valid	114
Terminology.....	115
9.2 OpenDDA Sets.....	116
Two types of sets.....	116
How variables are assigned to sets.....	116
The 'ALL' selection set	116
9.3 How Dynamic Selection of Data Works	117
The basic concept	117
The CIO file is not changed.....	117
9.4 Identifying Dynamic External Reference Components	118
Specifying a static External Reference.....	118
Specifying a dynamic External Reference.....	118
When the application is built	118
9.5 Alias Variable.....	119
Overview.....	119
syntax	119
Alias Variable examples	119
Using the FULL_NAME keyword	120
9.6 Loading Alias Variables.....	121
Introduction.....	121
Overview.....	121
Saving the CIO	121
Persistence of External References	121
External Reference components are strings	121
C examples	122
FORTRAN examples.....	122
Loading DDA_NULL	122
9.7 Optional EDB Variables	123
Overview.....	123
External Reference Status Variable	123
Modify Code Variable	124
9.8 Arrays and Indexes	125
Overview.....	125
Accessing multiple references.....	125
Accessing multiple values from an external array	126
9.9 MODIFY CIO Statement	127
Overview.....	127
Syntax.....	127
If Modify errors are encountered	127
If Alias Variables are NULL	127
Example	128
9.10 Optional EXEC DDA Statements.....	129
Overview.....	129
LOAD ALIAS statement.....	129
LOAD MODIFY_CODE statement.....	130
Usage	130

Table of Contents

10. CACHE MAINTENANCE TOOL	131
10.1 Overview.....	131
Background	131
Purpose of the cache.....	131
Purpose of the Cache Maintenance Tool	131
10.2 Characteristics of the Cache	132
The cache is memory mapped	132
Cache files.....	132
Cache size.....	132
If the cache is deleted.....	132
Error creating cache file.....	132
Cache entries	133
Cache entry freshness.....	133
10.3 Invoking the Cache Maintenance Tool	134
Invoking CMT from the shell prompt.....	134
Redirection and piping	134
10.4 Cache Maintenance Tool Commands.....	135
Command list.....	135
Options	135
Summary of commands.....	135
Qualifiers	136
external_reference_name.....	136
freshness_time	137
Checkpoint/ Restore	137
show cache command	138
Case of commands and qualifiers	138
Command line limits	139
Delete confirmation options	139
10.5 Examples	140
show entry examples	140
show freshness example	141
show cache example	142
delete entry examples.....	142
delete freshness example	143
checkpoint example	143
Restore example.....	143
11. OPENDDA APPLICATION EXAMPLES	145
11.1 Application Examples Distributed with the Software	145
Purpose of application examples.....	145
Location of application examples	145
Table of examples included	145
11.2 How to Build the ddademo Application.....	147
Introduction	147
FORTRAN and C programs provided	147
What the ddademo application does	147
11.3 Prepare ddademo	148
Copy files.....	148
Edit EDB section.....	148

Table of Contents

11.4 Build ddademo	150
How to build.....	150
Files created by build	150
Possible errors	151
11.5 Validate DEF File	152
How to validate.....	152
11.6 Compile/Link ddademo.....	153
Compile and link.....	153
11.7 Run ddademo Offline.....	154
Try to run offline	154
If security is set to read/write.....	154
If security is NOT set to read/write	154
11.8 Install ddademo.....	156
Install your ddademo application.....	156
11.9 Sample File Listings	157
ddademo listing	157
ddademo.def listing	160
12. OPENDDA DEVELOPMENT GUIDELINES AND STANDARDS	161
12.1 Overview	161
Purpose	161
Prerequisite	161
AIA framework	161
Scope	161
Honeywell SDK's	161
References	162
12.2 Applications.....	163
Definition	163
12.3 Application Enablers	164
Definition	164
Function.....	164
OpenDDA product description.....	164
Platform	164
12.4 FORTRAN 77 Standards.....	165
Standard FORTRAN 77.....	165
HP-UX FORTRAN 90 compiler.....	165
12.5 FORTRAN Extensions and Programming Techniques	166
Acceptable extensions	166
Argument matching	166
Recursion	166
Passing different size data types.....	166
IMPLICIT NONE	166
Array subscripts.....	166
12.6 Information on C/C++.....	167
Reference for C/ C++ information	167

Table of Contents

13.	OPENDDA COMPILE AND LINK SCRIPTS	169
13.1	C Compile and Link Script File.....	169
Filename		169
Script listing		169
Debug mode script		169
Makefile		169
13.2	C++ Compile and Link Script File.....	170
Filename		170
Script listing		170
Debug mode script		170
13.3	FORTTRAN Compile and Link Script File.....	171
Filename		171
Script listing		171
Debug mode script		171
Makefile		171
14.	GLOSSARY	172

Tables and Figures

Table 1-1	OpenDDA User's Guide Sections.....	20
Table 2-1	Application Source Files.....	29
Table 3-1	"Make" and Script Files	36
Table 4-1	How to Create an OpenDDA Application.....	47
Table 4-2	How to Build an OpenDDA Application.....	49
Table 4-3	How to Build an Application when the EDB is Separate	52
Table 4-4	Move a Remote Node OpenDDA Application to an A ^x M.....	56
Table 4-5	How to Install an OpenDDA Application Online	58
Table 4-6	How to Execute an Offline OpenDDA Application on an A ^x M	60
Table 4-7	How to Execute an Offline Application on a Remote Node.....	62
Table 4-8	How to Execute an Online Application on an A ^x M	63
Table 5-1	OpenDDA Command Line Components	66
Table 5-2	How OpenDDA Builds the dda Command Line	68
Table 5-3	OpenDDA Command Line Qualifiers.....	69
Table 5-4	OpenDDA Command Line Options	70
Table 5-5	File Protections	88
Table 6-1	List of OpenDDA Application File Types.....	94
Table 7-1	OpenDDA Status Variables.....	98
Table 7-2	Facility Codes	99
Table 7-3	Meaning of Message Number	100
Table 7-4	Severity Values	100
Table 7-5	Location of Status Components	102
Table 7-6	Actions for each Severity Level	104
Table 9-1	Valid Location Specifications for Dynamic Selection	114
Table 10-1	Default Qualifier Values	136
Table 11-1	Application Examples Included with OpenDDA	145
Figure 4-1	Create a New OpenDDA Application.....	46
Figure 4-2	How to Build an OpenDDA Application	48
Figure 4-3	Building an OpenDDA Application with a Separate EDB	51
Figure 4-4	Moving an OpenDDA Application from a Remote Node to an A ^x M.....	55
Figure 4-5	Installing an OpenDDA Application Online	57
Figure 4-6	Executing an OpenDDA Offline Application from an A ^x M	59
Figure 4-7	Executing an OpenDDA Offline Application from a Remote Node.....	61
Figure 4-8	Executing an OpenDDA Online Application from an A ^x M	62
Figure 5-1	Redirecting Standard Output.....	72

Acronyms

A ^x M	Application Module ^x (Application Module with Extensions)
DAT	Digital Audio Tape
LAN	Local Area Network
LCN	Local Control Network
PIN	Plant Information Network
SAM	System Administration Manager
TDC	Total Distributed Control
US	Universal Station
U ^x S	Universal Station ^x (Universal Station with Extensions)
WSI	Workstation Interface
TPS	TotalPlant Systems

References

Publication Title	Publication Number	Binder Title	Binder Number
<i>Application Module^x Customer Release Guide</i>	AX04-300	Application Module ^x	3094
<i>Application Module^x User Guide</i>	AX09-300	Application Module ^x	3094
<i>Application Module^x Troubleshooting</i>	AX13-300	Application Module ^x	3094
<i>Application Module^x Service</i>	AX13-410/510	Application Module ^x	3094
<i>LCN System Installation</i>	SW20-500	LCN Installation	3025

Hewlett-Packard CD-ROM Documentation

Hewlett-Packard Documentation at <http://www.docs.hp.com>

1. Using This Guide

1.1 Overview

Introduction

This guide explains what Open Data Definition and Access (OpenDDA) can do and how to use it to implement your OpenDDA applications.

OpenDDA is a versatile system and the conventions and organization used in this guide are designed to enhance your experience with OpenDDA.

What's in this section

This section contains:

- a review of this guide's organization,
- the conventions used, and
- a description of the sections and appendices of this guide.

Tutorial

Section 11 of this guide contains a brief tutorial using the supplied sample OpenDDA application - `ddademo`. We recommend you review Section 4—“Developing OpenDDA Applications” before you try the tutorial.

1.2 About This Document

Purpose

The purpose of this guide is to explain how to use the OpenDDA command line functions effectively with your OpenDDA applications so you can implement a wide range of advanced control solutions.

Audience

This guide is intended for process control engineers and software developers.

Software release

The information in this guide applies to OpenDDA Release 300.

Organization of this guide

The sections and topics of this guide are arranged to lead you through the steps required to build, test, and implement OpenDDA applications.

For your convenience, a glossary and an index are located at the end of the guide.

Tutorial

A.2 “How to Build the ddademo Application” in this guide contains a tutorial that takes you through the steps of building and executing an OpenDDA application.

We recommend you review Section 4—“Developing OpenDDA Applications” before you try the tutorial.

OpenDDA application reference information

This guide reviews the basics of OpenDDA. For application-specific information required to support a particular type application, refer to the OpenDDA Reference Manual.

Troubleshooting OpenDDA

For assistance in troubleshooting OpenDDA, refer to “Troubleshooting” in the OpenDDA Reference Manual.

1.3 Conventions Used in This Guide

Typed/Displayed text

Information messages displayed by OpenDDA, or information entered by a user, is indicated by this typeface:

```
EXEC DDA INITIALIZE STATUS = exec_status;
```

(example of user-entered application information)

```
Begin processing EDB...
```

```
DDA-S, EDB processing completed successfully.
```

(example of OpenDDA information message)

Upper/lowercase letters

In OpenDDA application examples, uppercase text is used to indicate information items as they *must* be entered:

```
C  EXTERNAL DATA BEGIN
```

Lowercase text is used to indicate information items (variables) whose names you choose:

```
application_name
```

ATTENTION

Command line options (qualifiers) are case-sensitive. High-level language variables in C or C++ are case-sensitive. Other elements of OpenDDA are not case-sensitive.

Optional parameters indicators

Bracket symbols [] are used in some examples to indicate optional parameters.

Example:

```
value_variable = test_value [/data_type] [/[NO]TEST_VALUE]
```

Applications

Throughout this guide, the term “application” refers to applications created using OpenDDA, unless otherwise stated.

1.4 Overview of the Sections in This Guide

Document sections and descriptions

The following table contains a brief description of the sections in this guide.

Table 1-1 OpenDDA User's Guide Sections

Section Name	Description
Using This Guide	The section you are reading now. It explains the conventions and organization of the OpenDDA User's Guide, this document.
Introduction to OpenDDA	Reviews the functions and features of the Open Data Definition and Access (OpenDDA) software package.
Preparing to Use OpenDDA	Describes how to configure your personal environment to work with OpenDDA.
Developing OpenDDA Applications	Discusses how to create new applications from supplied templates or from existing applications.
OpenDDA Application Development Functions	Contains descriptions and examples of the OpenDDA command line functions.
File Management	An OpenDDA application consists of many files. This section includes a listing and description of the OpenDDA file extensions.
Using Generic Status Codes	Explains how to use Generic Status Codes in your OpenDDA applications to have platform independent status codes returned when accessing external data.
Remapping CIO Files	Explains, at an overview level, the Remapping CIO functionality. This functionality allows an application to select among multiple CIO files at run time.
Dynamic Selection Sets	Explains, at an overview level, Dynamic Selection Sets functionality. This functionality allows an application to modify external data point names, parameter names, and/or index names at run time.
Cache Maintenance Tool	Describes the functionality of the External Reference cache and describes use of the cache maintenance tool.
Section 11 – How to Build the ddademo Application	Contains a brief tutorial using the supplied sample OpenDDA application - ddademo.
Section 12 – OpenDDA Development Guidelines and Standards	Provides additional information on the development of OpenDDA applications.

2. Introduction to OpenDDA

2.1 Overview

Introduction

OpenDDA (Data Definition and Access) is a platform independent application builder that provides you with a simple standard method of defining and accessing external data located on a Distributed Control System (DCS).

OpenDDA offers you endless flexibility to monitor and control data from your host platform.

DDA Host Platform support

The Host Platform is a hardware and software computer system that serves as an application development environment for OpenDDA. Host Platforms supported by OpenDDA include:

- Honeywell's Application Module^x (A^xM), which includes Hewlett-Packard's PA-RISC coprocessor running HP-UX operating system version 9.05 or later.
- A Hewlett-Packard workstation based on Hewlett-Packard's PA-RISC coprocessor running HP-UX operating system version 9.05 or later.

OpenDDA Applications

OpenDDA (Data Definition and Access) is a software system that provides assistance for developing and implementing applications written in

- FORTRAN
- C/C++

OpenDDA's Builder functions

OpenDDA provides extensive tools for:

- developing and testing offline applications and
- installing and operating online applications.

OpenDDA applications can range in size and complexity from unit operations up to entire plants.

OpenDDA's Executive functions

OpenDDA accomplishes the data transfer (input and output) for your applications to

- DCS data
 - For example: TPS point.parameter data

How OpenDDA operates

OpenDDA operates by interfacing one or more Host Platform software applications to communicate with a DCS.

OpenDDA requires you know

To use OpenDDA effectively, you do not need to be a sophisticated computer user. The only computer knowledge that you really need is how to “log in,” how to use a text editor, and how to use the application’s implementation tools.

Most applications will exchange data with a DCS, so you will need a basic understanding of the DCS, particularly building data points and perhaps schematic displays.

You need to understand

- point building and the use of schematics on the DCS
- the application that is being developed
- and the functions of the OpenDDA components and how they are used.

2.2 What Can OpenDDA Do For You?

Major features of OpenDDA

The major features of OpenDDA include

- Real-time scheduling of applications
- Access to real-time process data
- Easy access to the DCS database
 - A connection with the DCS database is automatically created.
- Easy, straightforward testing setup
 - You can debug your applications with test value substitution and toggle between test and live data.
- High Level Language (HLL) programming support
 - FORTRAN
 - C/C++
- OpenDDA execution statements
 - You can specify special actions within your OpenDDA applications.
- Two techniques that allow you to dynamically select External References at run time
 - Remapping CIO Files
 - Dynamic Selection Sets
- Hibernation functionality
 - Allows an application to maintain data values between executions (persistence)
 - Reduces startup overhead when an application executes repeatedly

2.3 Introduction to OpenDDA Applications

What is an OpenDDA application?

An OpenDDA application performs an overall function, such as the calculation of operating information for a particular process or optimization of a particular plant.

An OpenDDA application may be utilized by the following application types:

- FORTRAN
- C/C++

OpenDDA imposes no limit on the number of applications that can be developed on a system.

Offline/Online State

An application is in the *offline* state when you create it, and it remains offline until you install it *online*.

An application is in the *online* state when it has been installed using OpenDDA.

2.4 OpenDDA Offline Applications

Offline directory

The offline directory is any directory in which you choose to develop your OpenDDA applications.

Offline applications

OpenDDA applications are considered "offline" when they are applications which are being

- developed,
- tested, and
- debugged.

Offline applications can

- execute only on demand,
- be modified at any time, and
- write to the DCS only if the security switch is set to READWRIT.

Offline applications cannot

- be initiated from CL.

2.5 OpenDDA Online Applications

Online directory

The Online Directory is a special directory (/users/axm) that contains a link to your OpenDDA application, enabling a Control Language (CL) application to initiate your OpenDDA application.

Online applications

Online applications are applications that were developed using the OpenDDA Development Functions including the Install Application function. Online applications are applications which are

- fully developed,
- fully debugged, and
- ready to execute.

Online applications

- can be scheduled for periodic execution
- can be executed on demand from a DCS or from an OpenDDA Host Platform
- can read and write data outside the application
- can be located in a secure directory where they cannot be modified, and
- can hibernate to accommodate memory resident applications and maintain data persistence between executions.

2.6 Data Access

What is data access?

OpenDDA applications communicate with the process and with process operators and engineers through the distributed control system (DCS) database.

A large OpenDDA application can access thousands of data items.

DCS data

OpenDDA applications use the DCS database as their primary source of data. Using the DCS database has several important advantages. It can include all process data needed by the application.

Outputs to the process can follow standard cascade rules, under control of the operator or lower-level programs in an Application Module^x (A^xM).

- Additional data required by the application and data generated by the application can be stored in point parameters in the DCS. It's very easy to add new data points and parameter types to accommodate new applications.
- Data can be included in DCS custom displays. Using the same console that they use to monitor and control the process, process operators and engineers can monitor what an application is doing and control its operation.
- OpenDDA applications can access a variety of data types:
 - real
 - enumerated
 - integer
 - time
 - logical/boolean
 - external IDs
 - string
 - internal IDs
- Your application's native language (FORTRAN or C/C++) variables are automatically populated and/or depopulated with DCS data whenever you desire.

2.7 Introduction to OpenDDA Application Types

OpenDDA application scope

The models and programs that you develop can range from small and straightforward to large and complex, depending on the application's type and purpose.

Whatever the application, OpenDDA makes implementation of your application as simple as possible.

High Level Languages (HLL)

FORTRAN, C, or C++ can be used to develop custom advanced control applications.

OpenDDA applications are useful in situations that require logical decisions or involve transferring process conditions data, such as coordination of recipe downloading for large batch operations.

2.8 Application Source and External Data

Application source files

All applications have a *source* file that determines what the application does. The table below describes the source files for each application type.

Table 2-1 Application Source Files

Application Type	Source File Description
FORTRAN	Consist of user-generated FORTRAN statements Reference: Section 8—"FORTRAN Applications" in the <i>OpenDDA Reference Manual</i> .
C/C++	Consist of user-generated C statements Reference: Section 9—"C Applications" in the <i>OpenDDA Reference Manual</i> .

External data

OpenDDA External Data Blocks (EDB) define what data is to be input and output from outside the application. Such data is called *external* data and includes the following:

- Real-time process data from DCS Systems

3. Preparing to Use OpenDDA

3.1 Overview

Introduction

Before you begin developing OpenDDA applications you should review this section to acquaint yourself with the OpenDDA directory structure and the steps you need to take for the most efficient use of OpenDDA:

- establishing the OpenDDA command – DDA
- example files to help you create OpenDDA applications
- how to use OpenDDA environment variables with your applications
- how OpenDDA implements security.

Tutorial

A.2 “How to Build the ddademo Application” in this guide contains a brief tutorial using the supplied sample OpenDDA application - ddademo.

We recommend you review Section 4—“Developing OpenDDA Applications” before you try the tutorial.

3.2 OpenDDA Directory Structure

Directory tree

/opt | DDA

bin	doc	examples	include	lib
dda	dda.help	alltestf.def	dda_fortran.f	libddagdx.sl
dda_xstatus		alltestf.f	hgsc_fortran_errors.f	libddagen.sl
dda_template.def		alltestc.def	hgsc_c_errors.h	libdda.sl
dda_system.def		alltestc.c	dda_fortran_errors.f	
fortran.f		lvrf.def	dda_c_errors.h	
c.c		lvrf.f	dda_c.h	
cmt		dda_create_fortran_debug_executable		
		dda_create_fortran_executable		
		dda_fortran_examples.make		
		dda_c_examples.make		
		dda_create_c_debug_executable		
		dda_create_c_executable		
		dda_create_cpp_debug_executable		
		dda_create_cpp_executable		
		ddademo.def		
		ddademo.f		
		ddademo.c		
		event_hiber.c		
		readit.def		
		readit.f		
		read_tank_info.c		
		remap.c		
		remap2.edb		
		remap3.edb		

Directory contents

The Directory...	Contains...
bin	executables and templates
doc	online help
examples	useful examples including: • OpenDDA applications • make files • script files
include	OpenDDA application include files
lib	OpenDDA shared libraries

3.3 The OpenDDA Command

Preparing for the OpenDDA command

Before you can use the OpenDDA system, the command “dda” must be made “visible” to your login process (your terminal session).

Check with your system administrator. They may have included the “dda” command as visible to all users. If so, you can skip the following step.

To enable the OpenDDA command

The following command defines a logical name “dda” to point to the executable program found in the /opt/DDA/bin directory:

```
alias dda '/opt/DDA/bin/dda'
```

Placing the previous command line in your .login file automates the loading of OpenDDA.

csch command

The command shown above works in csch (C shell). You may need to use a different command if you are using a different UNIX shell.

ATTENTION

If your system administrator has not included the “dda” command as visible to all users, you must enter this command each time you log in to the Host Platform and intend to use OpenDDA.

3.4 Example Files

OpenDDA examples provided

OpenDDA provides an assortment of examples to help you develop OpenDDA applications. You may want to examine some or all of these before building an application from scratch.

Template starter files

Template files are provided to help you start developing OpenDDA applications. These template files contain all the basic ingredients necessary for an OpenDDA application. (Refer to 4.2 for more details.) The two files needed to create a new FORTRAN OpenDDA application are

`fortran.f`

`dda_template.def`

The two files needed to create a new C or C++ OpenDDA application are

`c.c`

`dda_template.def`

These template files are located in the directory

`/opt/DDA/bin`

Example OpenDDA applications

OpenDDA application examples are provided to give you

- samples to learn about OpenDDA and
- examples you can copy to your own private directory and then modify for your own needs.

The OpenDDA application examples are located in the directory

`/opt/DDA/examples`

“Make” and script files

Several example “make” files are available for you to examine or copy and modify for your own use. They are in the directory

`/opt/DDA/examples`

Note: These make files were changed for OpenDDA R200 to use shared libraries. If you modify a make file, be sure to use one supplied with R200 or later. If you previously modified an R100 or R110 make file, you will have to make additional changes to accommodate shared libraries.

This table identifies the make and script files and explains what they do.

Table 3-1 “Make” and Script Files

Filespec	Purpose
<code>dda_fortran_examples.make</code>	This file acts upon each of the OpenDDA FORTRAN example applications provided in the <code>/opt/DDA/examples</code> directory. It does everything necessary to make an OpenDDA Application that will access test values. (Refer to the instructions inside this script file on how to use it.)
<code>dda_create_fortran_executable</code>	This general purpose script file compiles and links a FORTRAN application already developed using the OpenDDA Application Development Functions.
<code>dda_create_fortran_debug_executable</code>	This general purpose script file compiles and links a FORTRAN application already developed using the OpenDDA Application Development Functions so that it can be executed under control of the symbolic debugger.
<code>dda_c_examples.make</code>	This file acts upon each of the OpenDDA C example applications provided in the <code>/opt/DDA/examples</code> directory. It does everything necessary to make an OpenDDA Application that will access test values. (Refer to the instructions inside this script file on how to use it.)
<code>dda_create_c_executable</code>	This general purpose script file compiles and links a C application already developed using the ANSI C compiler and the OpenDDA Application Development Functions.

Table 3-1 “Make” and Script Files

Filespec	Purpose
dda_create_c_debug_executable	This general purpose script file compiles and links a C application already developed using the ANSI C compiler and the OpenDDA Application Development Functions so that it can be executed under control of the symbolic debugger.
dda_create_cpp_executable	This general purpose script file compiles and links a C++ application already developed using the ANSI C++ compiler and the OpenDDA Application Development Functions.
dda_create_cpp_debug_executable	This general purpose script file compiles and links a C++ application already developed using the ANSI C++ compiler and the OpenDDA Application Development Functions so that it can be executed under control of the symbolic debugger.

3.5 Shared Libraries

Description

OpenDDA R300 uses shared libraries as opposed to the archive libraries that were used in R100 and R110. With shared libraries, object code from a library is not included in the application executable. Instead, the linker inserts a reference to the shared library. Then, at program execution, the dynamic loader loads any required shared libraries into the process's address space and resolves the addresses of the routines in the shared libraries.

Advantages

The shared libraries used in OpenDDA have the following advantages:

- Conserve disk space and virtual memory requirements
- Allow OpenDDA to have execution level compatibility for migration if no new functions are used; that is, users will not need to recompile or relink their applications during migration

Location

Shared libraries are in the directory `/opt/DDA/lib` and have the extension `.sl`. Once an executable is created, the library must not be moved because the executable contains the absolute pathname of the library.

Impact on users

After an upgrade to OpenDDA R200 or R300, existing applications that were generated with the R100 and R110 archive libraries will continue to run as before. However, if they are recompiled and relinked, they will use the shared libraries. This will be transparent to users who use the supplied make scripts.

If you create your own make scripts, you should modify the R200 version to pick up the proper options for shared libraries.

3.6 Compiler Options

Recommended compiler options for FORTRAN

If you create your own script file to compile/link your OpenDDA applications, we strongly recommend you use the following compiler options for FORTRAN applications:

- C Enables range checking
- u Forces types of identifiers to be explicitly declared. (This has the same effect as specifying IMPLICIT NONE in your source file.)

Suggested compiler option for FORTRAN

In addition to the options listed above, we suggest you use the following option:

- w Suppresses warning messages

If you do not use this option, you may get warning messages similar to the following when you compile your OpenDDA application:

```
Warning on line 57 of appl_src.f: local variable dda_loop_control never used
```

```
Warning on line 57 of appl_src.f: local variable dda_read_data never used
```

```
Warning on line 57 of appl_src.f: local variable dda_write_data never used
```

```
Warning on line 57 of appl_src.f: local variable dda_dummy_timestamp never used
```

```
Warning on line 57 of appl_src.f: local variable dda_term_readpop_timer never used
```

```
Warning on line 57 of appl_src.f: local variable dda_term_writedepop_timer never used
```

```
Warning on line 57 of appl_src.f: local variable dda_term_writepop_timer never used
```

```
Warning on line 57 of appl_src.f: local variable dda_init_popdepop_timer never used
```

Any of these warnings can safely be ignored.

Track compiler errors

If you get errors when you compile your application, you may want to add the following option to your compile:

- V Writes a program listing to a .l file (e.g., myapp.l). This will help you track down your compiler errors.

Recommended compiler options for C

If you create your own script file to compile/link your OpenDDA applications, we strongly recommend you use the following compiler options for C applications:

- Z Allows dereferencing of null pointers
- Aa Compile under ANSI mode

Recommended compiler options for C++

If you create your own script file to compile/link your OpenDDA applications, we strongly recommend you use the following compiler options for C++ applications:

- Z Allows dereferencing of null pointers
- Aa Request ANSI mode HP C++ preprocessor
- a1 Causes the translator/compiler to produce ANSI C-type declarations
- Os Suppress optimizations for HP-UX C++ compiler version A.03.20
(Capital letter O followed by small letter s)
or
- +O0 Suppress optimizations for HP-UX C++ compiler version A.03.50 or
later (Capital letter O followed by zero)

3.7 Environment Variables

OpenDDA environment variables

OpenDDA utilizes some environment variables to make it more convenient for you to develop and execute OpenDDA applications. You may want to define a default value for these environment variables in

- your .login script file and
- special-purpose script files you define to automate OpenDDA application development.

DDAOPTS

DDAOPTS is a special environment variable used by the OpenDDA command line functions. It can be used to pass information you define to the DDA command line for your convenience.

Refer to “Using DDAOPTS” in 5.2 for detailed information on how to use it.

DDA_REDIRECT_OUTPUT

By default, OpenDDA applications redirect their output to the OpenDDA Application’s General Output file.

DDA_REDIRECT_OUTPUT is interrogated when executing OpenDDA applications offline to determine whether their output should be directed to the Standard Output or to the OpenDDA General Output file.

The following table illustrates how an Offline OpenDDA Application’s output is directed depending upon the value of DDA_REDIRECT_OUTPUT (shown in " "):

Value of DDA_REDIRECT_OUTPUT	Output Destination
"n"	Standard Output
"N"	Standard Output
anything other than "n" or "N"	OpenDDA General Output file

Use your UNIX shell’s environment command to set the DDA_REDIRECT_OUTPUT environment variable’s value.

3.8 A^xM Security

Security Switch

The Application Modulex has three settings for the security switch on the LCN:

- READONLY
- RW_LCN_I
- READWRIT

An OpenDDA application will execute differently depending on the security switch setting.

READONLY

When the security setting is READONLY, applications will successfully read the INPUT values declared in the External Data Block (EDB). Values declared as OUTPUT in your STORE_STATUS variables will contain the following error:

```
393322 - SYS-E, Permission denied
```

RW_LCN_I

When the security setting is RW_LCN_I, applications installed in the /users/axm directory and executed by a CL program (CL-initiated application) will be able to read and write data to the LCN. Non-CL initiated applications will function as READONLY applications.

READWRIT

When the security setting is READWRIT, all applications executed on the AxM have full read/write access of LCN data.

If your AxM is part of a network, setting the security switch to READWRIT does NOT grant access to applications executed from the Plant Information Network (PIN).

A^xM and OpenDDA file protections

OpenDDA is installed with the GROUP and OWNER protections set to **axm**. This default can be modified by your system administrator to be either more or less secure and should be changed on all files below the

`/opt/DDA` directory tree.

ATTENTION

The OpenDDA installation does not affect the protections set on the `/opt` directory if the directory already exists. Your system administrator may need to modify the GROUP and OWNER of the `/opt` directory to **axm**, before A^xM users can access the `dda` executable.

4. Developing OpenDDA Applications

4.1 Overview

Introduction

To begin application development, you start by

- creating a new application or
- copying an existing one that you may wish to modify.

This section discusses both methods of creating applications.

Throughout this section, the example application being developed is named myapp.

Tutorial

Subsection 11.2, contains a brief tutorial that takes you through the steps to build ddademo, the sample OpenDDA application supplied with your software.

The application listing and its .def file are also included.

We recommend you review this section before you try the tutorial.

4.2 How to Create a New OpenDDA Application

Creating a new OpenDDA application

The following figure illustrates how to create an OpenDDA application. The figure illustrates the creation of a FORTRAN example. To create a C application, copy the templates `dda_template.def` and `c.c` to create the files `myapp.def` and `myapp.c`.

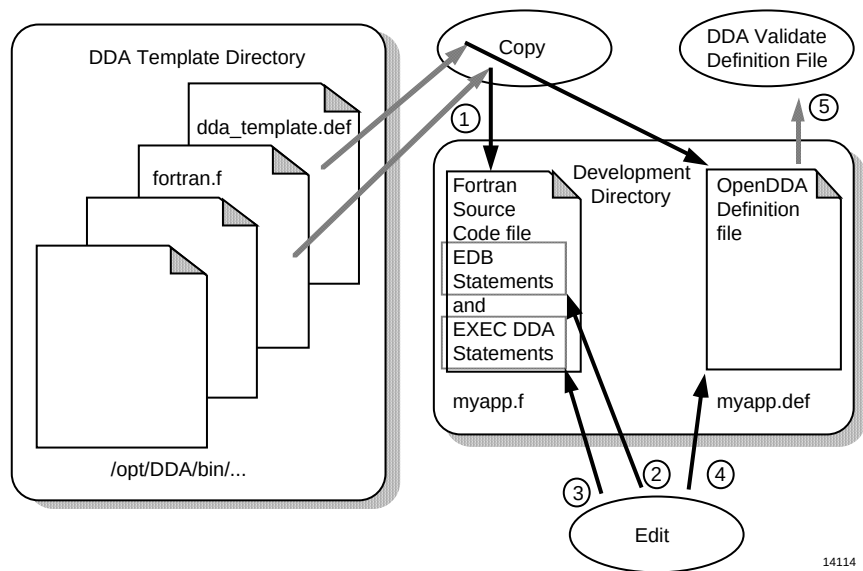


Figure 4-1 Create a New OpenDDA Application

How to create an OpenDDA application

The following table explains how to create an OpenDDA application.

Table 4-1 How to Create an OpenDDA Application

Step	Action	Result	Reference
1	Use your Host Platform copy function to copy the file: <code>fortran.f</code> to <code>myapp.f</code>, or <code>c.c</code> to <code>myapp.c</code> Example: <code>cp /opt/DDA/bin/fortran.f /users/mydir/myapp.f</code> and copy (optional*): <code>dda_template.def</code> to <code>myapp.def</code> Perform the copy from the <code>/opt/DDA/bin</code> directory to a working directory of your own. Use a custom filename for these files for this example, we use the filename "myapp". You should use the same file extensions. TIP You can copy existing OpenDDA applications and customize them.	You create the only file or files you need to modify to specify the behavior of your OpenDDA application.	Your Host Platform copy function
2	Customize the External Data Block (EDB) inside the <code>myapp.f</code> or <code>myapp.c</code> file. TIP Remember, the declaration for the native variables referred to in the EDB need to be inside the <code>myapp.f</code> or <code>myapp.c</code> file.	You define an association between native variables (FORTRAN or C) and External Data.	3.2 in the <i>OpenDDA Reference Manual</i>
3	Insert EXEC DDA statements at the appropriate places inside your <code>myapp.f</code> or <code>myapp.c</code> source code. TIP You must have the EXEC DDA INITIALIZE (as the first EXEC DDA statement executed) and EXEC DDA TERMINATE (as the last EXEC DDA statement executed) in your application.	You define the places inside your source code that result in External Data transfers.	Section 6 in the <i>OpenDDA Reference Manual</i>
4	Customize the <code>myapp.def</code> file (optional*).	The runtime behavior of your application is defined.	Section 7 in the <i>OpenDDA Reference Manual</i>
5	Use the DDA Validate Definition File function (optional*) to validate <code>myapp.def</code>. TIP Any errors detected by any OpenDDA Application Development Functions will be directed to Standard Output. Usually this means the output will be displayed to a terminal, but you can redirect it to the output file of your choice.	The syntax of <code>myapp.def</code> is checked for accuracy.	5.12 Host Platform Standard Output redirection
You are now ready to build your Open DDA Application.			4.3

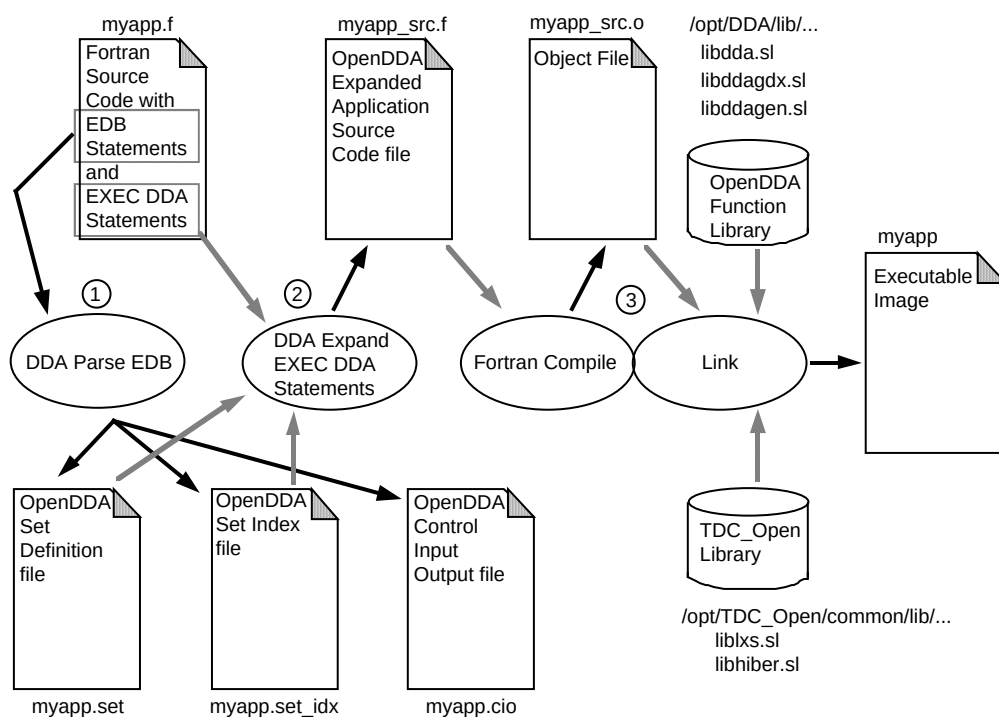
* The `.def` file is not required. Refer to Section 7 of the *OpenDDA Reference Manual*

4.3 How to Build an OpenDDA Application

Building an OpenDDA application

The following figure illustrates how to build an OpenDDA application. The figure shows a FORTRAN application. For a C application, the source file would be `myapp.c` instead of `myapp.f`.

In this example, the EDB is included in the application source code. For an example in which the EDB is in a separate file, see the next subsection.



14365

Figure 4-2 How to Build an OpenDDA Application

How to build an OpenDDA application

The following table explains how to build an OpenDDA application.

Table 4-2 How to Build an OpenDDA Application

Step	Action	Result	Reference
1	Use the OpenDDA Parse External Data Block function to parse <code>myapp.f</code> or <code>myapp.c</code>. TIP If your application is in C , you must use the <code>-Lc</code> option (specifies the C language.) Examples: <pre>dda -pe myapp.f (FORTRAN) dda -Lc -pe myapp.c (C)</pre>	Three OpenDDA files are created: <pre>myapp.set myapp.set_idx myapp.cio</pre> The <code>myapp.cio</code> file will be used at execution time. The other two files will be used in Step 2.	5.9
2	Use the OpenDDA Expand EXEC DDA Statements function with <code>myapp.f</code> or <code>myapp.c</code>. TIP Never modify the <code>myapp_src.f</code> or <code>myapp_src.c</code> file. Whenever you need to make modifications, modify the <code>myapp.f</code> or <code>myapp.c</code> file and perform this step again. TIP If your application is in C , you must use the <code>-Lc</code> option (specifies the C language.) Examples: <pre>dda -ee myapp,f (FORTRAN) dda -Lc -ee myapp.c (C)</pre>	A new file: <pre>myapp_src.f or myapp_src.c</pre> is created that is an exact duplicate of <code>myapp.f</code> or <code>myapp.c</code> except the EXEC DDA Statements in <code>myapp.f</code> or <code>myapp.c</code> are replaced with OpenDDA function calls.	5.10

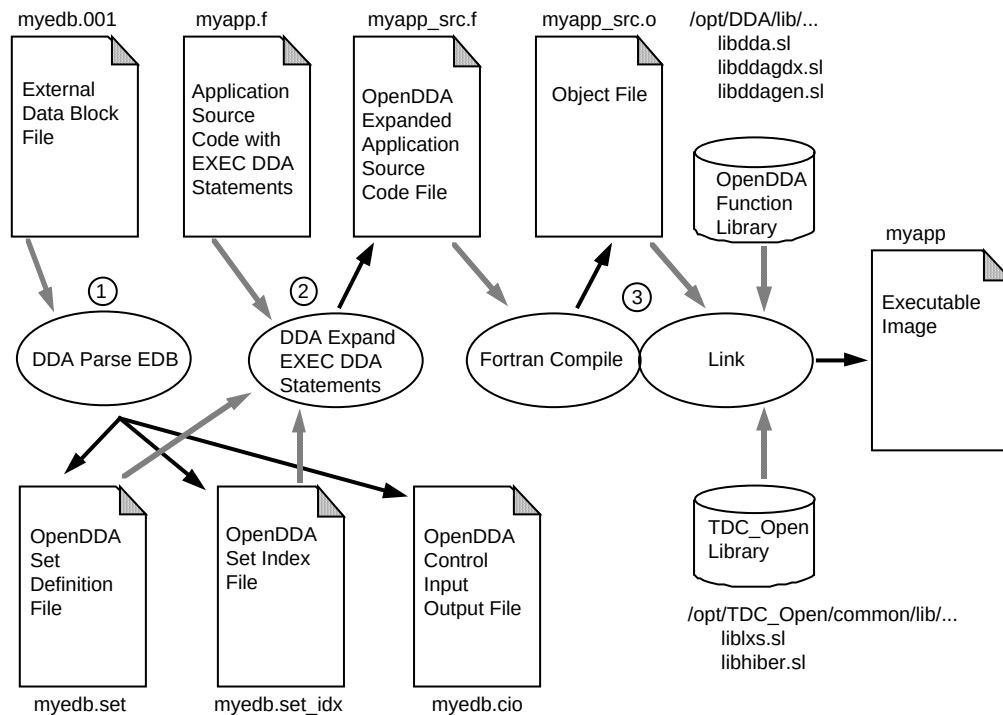
Table 4-2 How to Build an OpenDDA Application

Step	Action	Result	Reference
3	Compile and link the <code>myapp_src.f</code> or <code>myapp_src.c</code> file. Several files from two different libraries need to be linked into the executable image. The OpenDDA Function Library files are located in directory: <code>/opt/DDA/lib</code> The files to link to are: <code>libdda.sl</code> <code>libddagdx.sl</code> <code>libddagen.sl</code> The TDC_Open Library files are located in the directory: <code>/opt/TDC_Open/common/lib</code> The files to link to are: <code>liblxs.sl</code> <code>libhiber.sl</code> TIP Example compile and link script files are located in <code>/opt/DDA/examples</code>: <code>dda_create_fortran_executable</code> <code>dda_create_c_executable</code> <code>dda_create_cpp_executable</code> TIP You can compile and link this file in debug mode if you want to build an offline application to run under control of the debugger.	A <code>myapp_src.o</code> object file. A <code>myapp</code> executable image.	Compiler manual Linker manual OpenDDA Example linker instructions Debugger manual Section 13
You are now ready to: • execute your OpenDDA application offline, or • to install your Open DDA application to run online.			4.8 5.13

4.4 How to Build When the EDB is Separate

Building an application with separate EDB

The following figure illustrates how to build an OpenDDA application when the EDB is not imbedded in the application source file, but is in a separate file. The figure shows a FORTRAN application. For a C application, the source file would be `myapp.c` instead of `myapp.f`.



14366

Figure 4-3 Building an OpenDDA Application with a Separate EDB

How to build an application with separate EDB

The following table explains how to build an OpenDDA application.

Table 4-3 How to Build an Application when the EDB is Separate

Step	Action	Result	Reference
1	Use the OpenDDA Parse External Data Block function to parse <code>myedb.001</code> TIP If your application is in C, you must use the <code>-Lc</code> option (specifies C language.) Examples: <code>dda -pe myedb.001 (FORTRAN)</code> <code>dda -Lc -pe myedb.001 (C)</code>	Three OpenDDA files are created: <code>myedb.set</code> <code>myedb.set_idx</code> <code>myedb.cio</code> The <code>myedb.cio</code> file will be used at execution time. The other two files will be used in Step 2.	5.9
2	Rename the Control Input Output (CIO) file so that it has the same base name as the source file plus the extension <code>.cio</code> (optional). Example—to rename <code>myedb.cio</code> to <code>myapp.cio</code> , enter: <code>mv myedb.cio myapp.cio</code>	By renaming the CIO file as indicated, the EXEC DDA INITIALIZE function will automatically read the file. Otherwise, you will need to do an EXEC DDA READ CIO.	See the <i>OpenDDA Reference Manual</i> , subsections 6.9 and 6.15

Table 4-3 How to Build an Application when the EDB is Separate

Step	Action	Result	Reference
3	Use the OpenDDA Expand EXEC DDA Statements function with <code>myapp.f</code> or <code>myapp.c</code>. TIP Never modify the <code>myapp_src.f</code> or <code>myapp_src.c</code> file. Whenever you need to make modifications, modify the <code>myapp.f</code> or <code>myapp.c</code> file and perform this step again. TIP If you application is in C , you must use the <code>-Lc</code> option (specifies C language.) Examples: <pre>dda -ee -set myedb.set myapp.f (FORTRAN) dda -Lc -ee -set myedb.set myapp.c (C)</pre>	A new file: <code>myapp_src.f</code> or <code>myapp_src.c</code> is created that is an exact duplicate of <code>myapp.f</code> or <code>myapp.c</code> except the EXEC DDA Statements in <code>myapp.f</code> or <code>myapp.c</code> are replaced with OpenDDA function calls.	5.10
4	Compile and link the <code>myapp_src.f</code> or <code>myapp_src.c</code> file. Several files from two different libraries need to be linked into the executable image. The OpenDDA Function Library files are located in directory: <pre>/opt/DDA/lib</pre> The files to link to are: <pre>libdda.sl libddagdx.sl libddagen.sl</pre> The TDC_Open Library archive files are located in the directory: <pre>/opt/TDC_Open/common/lib</pre> The files to link to are: <pre>liblxs.sl libhiber.sl</pre> TIP You can compile and link this file in debug mode if you want to build an offline application to run under control of the debugger.	A <code>myapp_src.o</code> object file. A <code>myapp</code> executable image.	Compiler manual Linker manual OpenDDA Example linker instructions Debugger manual Section 13
You are now ready to: • execute your OpenDDA application offline, or • to install your Open DDA application to run online.			4.8 5.13

4.5 Developing an OpenDDA Application on a Remote Node

Creating an OpenDDA Application

The steps to create an OpenDDA application on a remote node are the same as creating one on an A^xM except for the parsing of the EDB. All external references must be in TEST mode when the EDB is parsed. This is done by either

- using the -T option on the dda command line (refer to subsection 5.3), or
- using the /TEST_VALUE option on all EDB statements and not using the -X option on the dda command line (refer to OpenDDA Reference Manual subsection 4.4).

Refer to subsections 4.3 and 4.4 for the steps required to develop an OpenDDA application.

4.6 Moving an OpenDDA Application Developed on a Remote Node to an A^xM

Moving an OpenDDA application

The following figure illustrates how to move an OpenDDA application developed on a Remote Node to an A^xM.

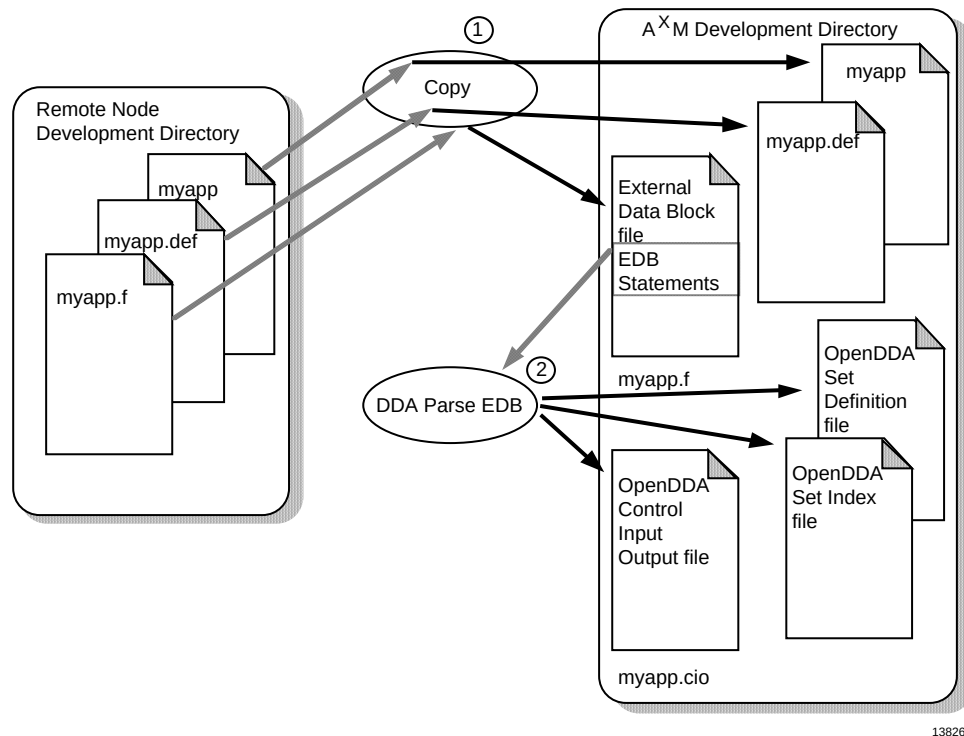


Figure 4-4 Moving an OpenDDA Application from a Remote Node to an A^xM

How to move an application from a remote node to an A^xM

The following table explains how to move an OpenDDA application developed on a Remote Node to an AxM.

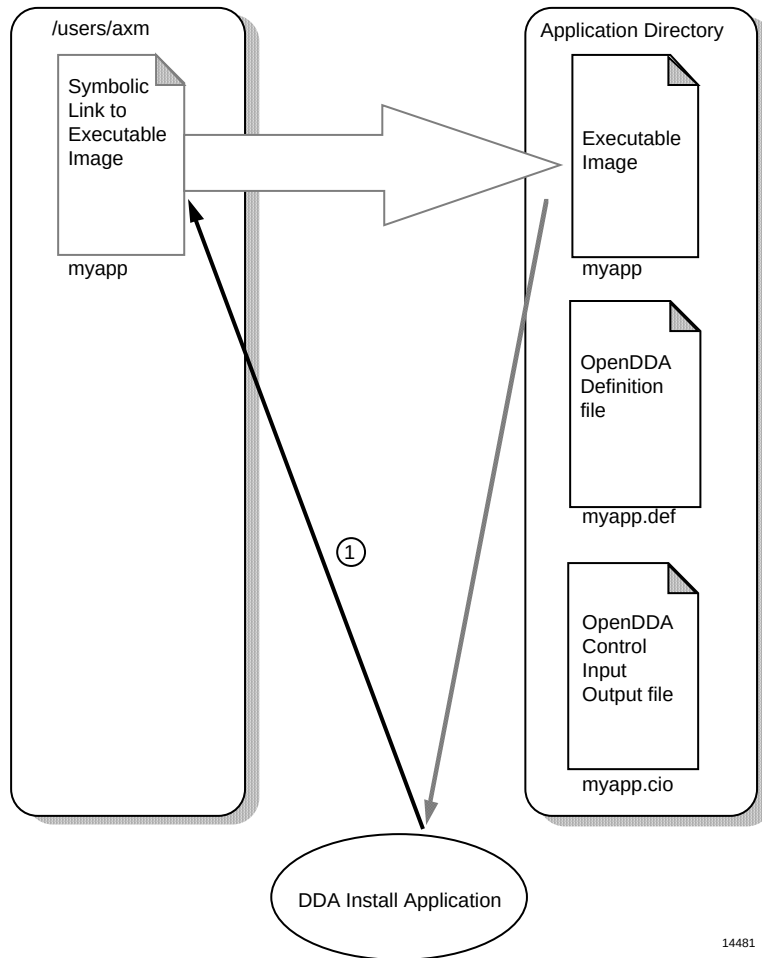
Table 4-4 Move a Remote Node OpenDDA Application to an A^xM

Step	Action	Result	Reference
1	Use your Host Platform copy function to copy the following files from your Remote Node development directory to an A^xM development directory: <code>myapp</code> <code>myapp.def</code> <code>myapp.f</code> or <code>myapp.c</code> TIP You are allowed to put your External Data Block into a separate file. In that case, use that file instead of the <code>myapp.f</code> or <code>myapp.c</code> file.	You are ready to validate your external data.	Host Platform copy function
2	Use the OpenDDA Parse External Data Block function to parse <code>myapp.f</code> or <code>myapp.c</code>. TIP You can put your External Data Block (EDB) into a separate file. In that case, use the name of that file instead of the <code>myapp.f</code> or <code>myapp.c</code> file. Examples: <pre>dda -pe myapp.f (FORTRAN) dda -pe -Lc myapp.c (C)</pre>	Three OpenDDA files are created: <code>myapp.set</code> <code>myapp.set_idx</code> <code>myapp.cio</code> The <code>myapp.cio</code> file will be used at execution time. The other two files are not needed.	5.9
You are now ready to: • execute your offline OpenDDA application, or • install the application online.			4.8 5.13

4.7 How to Install an OpenDDA Application Online

Installing an OpenDDA application online

The following figure illustrates how to install an OpenDDA application online.



14481

Figure 4-5 Installing an OpenDDA Application Online

How to install an application online

The following table explains how to install an OpenDDA application online.

Table 4-5 How to Install an OpenDDA Application Online

Step	Action	Result	Reference
1	Use the OpenDDA Install Application function to install myapp . Example: <code>dda -is myapp</code>	A symbolic link to: myapp is created in the <code>/users/axm</code> directory. The group axm execution privilege of the application is turned on.	5.13
2	Link a CL block that makes a subroutine (or function) call to the OpenDDA application to a point.	The point the CL block is linked to can now be used to initiate execution of the OpenDDA application, either on demand or a scheduled basis.	<i>CL/AM Reference Manual</i>
You are now ready to execute your OpenDDA online application.			4.8

Installed application execution behavior

When an installed application executes, the symbolic link of the executable file is used to create a path back to the Application Directory. This allows access to the application's supporting files (the CIO file and the optional DEF file) at initialization time, and supports creation of the General Output File (if required) back in the Application Directory.

How to execute an offline OpenDDA application on an A^xM

The following table explains how to execute an offline OpenDDA application on an A^xM.

Table 4-6 How to Execute an Offline OpenDDA Application on an A^xM

Step	Action	Result	Reference
1	Make sure you are in the directory that contains the OpenDDA application.	<code>myapp</code> will be loaded into memory. It will read the contents of the files:	Section 6
	From the Command Line, type in the name of the application: <code>myapp</code>	<code>myapp.def</code> <code>myapp.cio</code> .	3.7
	TIP You can redirect your output from the General Output file to Standard Output using the <code>DDA_REDIRECT_OUTPUT</code> environment variable. TIP Make sure the A ^x M Security Switch is set to the appropriate permission before attempting to write.	Then it will execute. External Data will be read and written as specified by the EXEC DDA Statements in <code>myapp.f</code> . Any general output will be directed to the file: <code>myapp.dda_out</code>	3.8

Executing an OpenDDA offline application from a Remote Node

The following figure illustrates how to execute an offline OpenDDA application on a Remote Node.

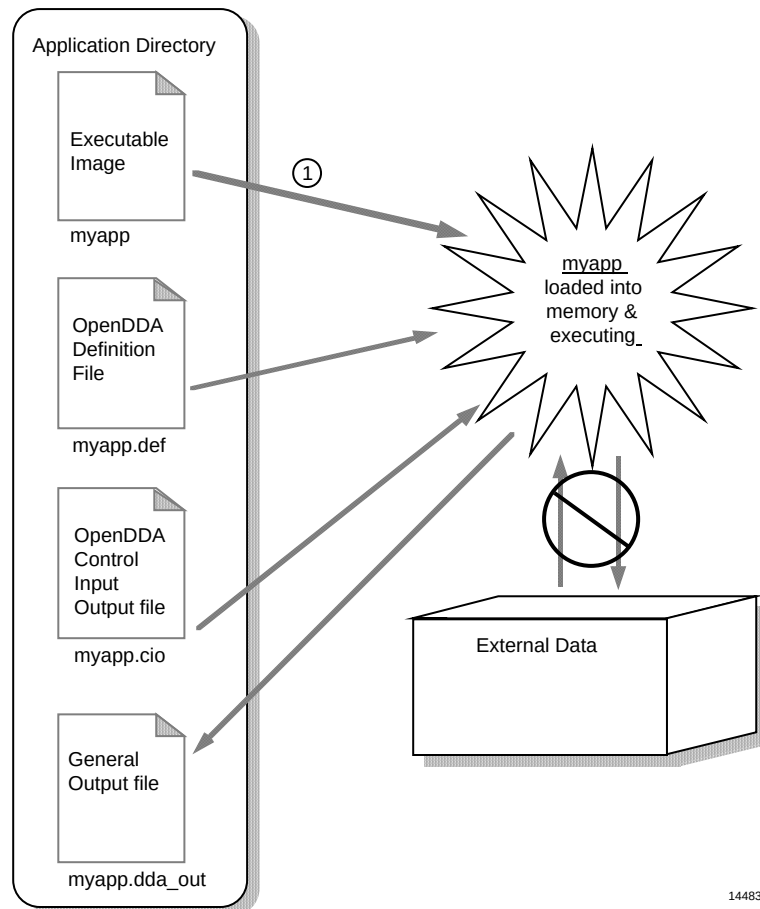


Figure 4-7 Executing an OpenDDA Offline Application from a Remote Node

How to execute an offline OpenDDA application on a remote node

The following table explains how to execute an offline OpenDDA application on a Remote Node.

Table 4-7 How to Execute an Offline Application on a Remote Node

Step	Action	Result	Reference
1	Make sure you are in the directory that contains the application. From the Command Line, type in the name of the application: myapp TIP You can redirect your output from the General Output file to Standard Output using the DDA_REDIRECT_OUTPUT environment variable.	myapp is loaded into memory. It reads the contents of the files: myapp.def myapp.cio. Then it executes. External Data is <i>not</i> read or written. Any general output will be directed to the file: myapp.dda_out	Section 6 3.7

Executing an OpenDDA online application from an AxM

The following figure illustrates how to execute an online OpenDDA application on an A^xM.

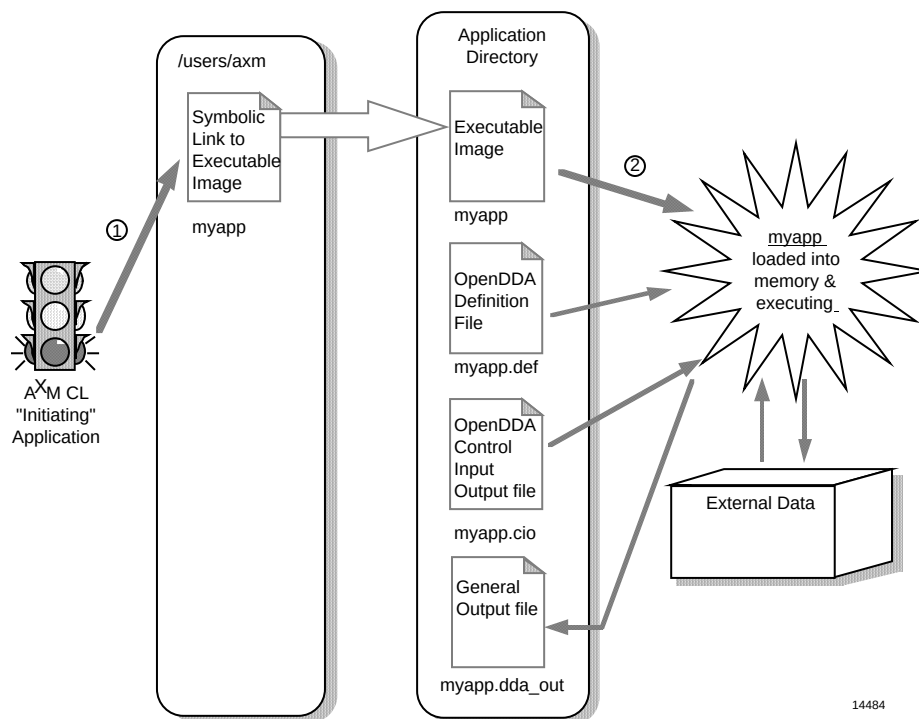


Figure 4-8 Executing an OpenDDA Online Application from an A^xM

How to execute an online OpenDDA application on an A^xM

The following table explains how to execute an online OpenDDA application on an AxM.

Table 4-8 How to Execute an Online Application on an A^xM

Step	Action	Result	Reference
1	Execute a CL block that makes a subroutine (or function) call to the OpenDDA application. TIP This can be accomplished either by setting the PPS parameter of the point to ON or by setting the PTEXECST parameter of an already scheduled point to ACTIVE.	The CL Application will initiate execution of the OpenDDA Application.	<i>CL/AM Reference Manual</i>
2	myapp is loaded into memory. TIP Make sure the AXM Security Switch is set to the appropriate permission before attempting to write.	myapp reads the contents of the files: myapp.def myapp.cio. Then it executes. External Data is read and written as specified by the EXEC DDA Statements in myapp.f or myapp.c	6.14 & 6.18 <i>OpenDDA Reference Manual</i>

5. OpenDDA Application Development Functions

5.1 Overview

Introduction

One way to create, build, install, and execute OpenDDA applications is to enter command strings at the DDA command line on your Host Platform. You can also use custom scripts to automate your commands.

DDA commands are made up of four possible components:

- the DDA command
- function qualifiers
- function options, and
- the filespec.

In this section

This section explains how to use the OpenDDA application development functions. These functions can be invoked through

- command line interface,
- script files, and
- an executing program.

5.2 Using OpenDDA Functions from the Command Line

OpenDDA command line interface

The OpenDDA functions are invoked through the command line interface of your Host Platform.

OpenDDA command line components

The OpenDDA command line consists of up to four components. The following table lists the components.

Table 5-1 OpenDDA Command Line Components

Component	Purpose	Example (in bold)
DDA command	Processes OpenDDA commands	dda
Qualifiers	Indicates functions to be performed	dda -pe
Options	Indicates an option of a function to be performed	dda -pe -o
Filespec Arguments	File Specification	dda -pe source_file.f

Composing an OpenDDA command line statement

The following rules apply to composing an OpenDDA command line:

- The **dda** command is always the first component.
- The **dda** command is always lowercase.

There must be one and only one qualifier.

- Qualifiers and options are preceded by a hyphen.
- Qualifiers and options must precede related filespec arguments.
- Qualifiers and options can appear in any order.
- One or more spaces must separate the dda command, qualifier, option, and filespec argument.
- Conflicting options will produce errors.

The DDAOPTS environment variable

DDAOPTS is a special environment variable used to pass information to the DDA command. The types of information passed to the DDA command are

- Qualifiers
- Options
- Filespec Arguments

Using DDAOPTS

You can use the DDAOPTS environment variable to customize the way your DDA command line is built.

Refer to your host platform documentation for specifics on how to set an environment variable. The following table illustrates how OpenDDA uses the DDAOPTS environment variable to build the DDA command line using the C shell.

This command...	Causes this to happen...	and has this result...
<code>setenv DDAOPTS=-o</code>	Sets the DDAOPTS environment variable	DDAOPTS now “contains” the character string “-o” DDAOPTS can now be seen and accessed by the dda command translator
<code>dda -pe hydro.f</code>	The dda command translates DDAOPTS into a character string and inserts that string in front of any command line qualifiers, options, or filespec arguments you enter.	The command line processed by the dda command will be: <code>dda -o -pe hydro.f</code>

How OpenDDA builds the dda command line

The following table illustrates how the dda command line is built.

Table 5-2 How OpenDDA Builds the dda Command Line

	If...	Then...
1	any explicitly entered command line qualifiers, options, or filespec arguments exist	they will be part of the command line.
2	the DDAOPTS environment variable is visible	the DDAOPTS character string is translated and inserted in front of the explicitly entered command line qualifiers, options, or filespec arguments (refer to “Using DDAOPTS”).
3	not all file extensions and potential options of the qualifier are accounted for	the defaults will be used. See 5.3 and 6.2.

5.3 Specifying OpenDDA Command Line Functions

Specifying OpenDDA functions

To specify the OpenDDA function to execute you need to combine the appropriate combination of qualifiers, options, and filespec arguments.

This subsection summarizes the available qualifiers, options, and filespec arguments. A detailed description of each function follows.

OpenDDA command line qualifiers

For each command line qualifier, the following table lists

- a brief description of the function
- a reference for more information, and
- whether it applies to offline or online applications.

Table 5-3 OpenDDA Command Line Qualifiers

Qualifier*	Description	Reference	Offline	Online
—cb	Complete Build	5.11	X	
—ds	Deinstall Application	5.14		X
—dv	Validate Definition File Syntax	5.12	X	
—ee	Expand EXEC DDA Statements	5.10	X	
—h	Help about dda command line functions and options	5.6	X	X
—is	Install Application	5.13	X	
—pe	Parse External Data Block	5.9	X	
*Note: the qualifiers are case sensitive .				

Filespec arguments

The following terms are used with *specific* meaning throughout this section:

Term	Description	Example (in Bold)
Filespec	The entire specification of a file	filename.extension
Filename	The portion of a filespec to the left of the leftmost period	filename .extension
File extension	The portion of a filespec to the right of the leftmost period	filename. extension filename_another. ext

Attention

A directory specification is not supported as part of a filespec. OpenDDA expects to find all filespecs in the current default directory.

OpenDDA command line options

For each command line option, this table lists

- a brief description of what the option indicates, and
- to which qualifiers the option can be applied.

Table 5-4 OpenDDA Command Line Options

Option*	Description	Used w/Qualifier(s)
-O	Overwrite existing output files. If this option is not used, OpenDDA prompts at the command line.	-cb, -pe, -ee
-S	Parse External Data Block syntax only - do not create ".set" and ".cio" files.	-pe, -cb
-set <i>filename.set</i>	Expand EXEC DDA Statements that relate to an External Data Block contained in a file other than the file containing the EXEC DDA Statements. Note: The -set option must be followed by the name of the ".set" file created by the Parse External Data Block function.	-ee
-T	Use Test Data - use the test values specified in the External Data Block when creating the ".set" and ".cio" files.	-pe, -cb
-X	Use External Data - use the external data references when creating the ".set" and ".cio" files.	-pe, -cb
-Lf, or -Lc	Specifies the language in which the application is written. Lf specifies FORTRAN, and Lc specifies C or C++. If neither is specified, FORTRAN is the default.	-pe, -ee, -cb
*Note: options are case-sensitive.		

Attention

If neither -X or -T are provided, OpenDDA will use whatever is explicitly specified in the EDB.

5.4 OpenDDA Command Line Messages

Viewing OpenDDA command line messages

OpenDDA command line messages consist of

- informational messages,
- error messages,
- completion status, and
- the number of errors encountered.

All OpenDDA command line messages are directed to Standard Output.

Redirecting Standard Output

If you prefer, you can redirect your Standard Output to a file. Then you can print the file or (if there are a large amount of messages) do automated searches, filtering, etc. If you don't care to see the output, you can redirect it to the null device. Refer to your Host Platform documentation for more information about redirecting Standard Output.

The following figure illustrates redirecting Standard Output.

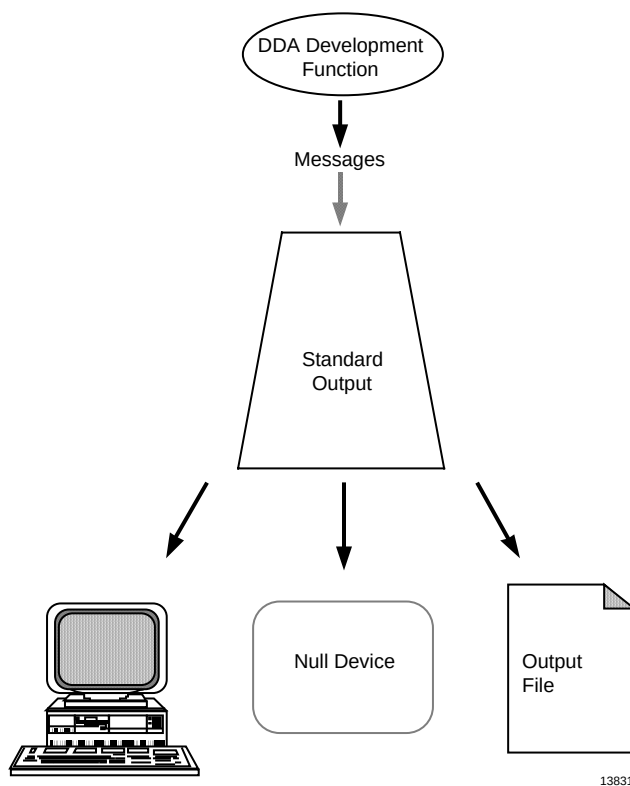


Figure 5-1 Redirecting Standard Output

Understanding OpenDDA command line error messages

OpenDDA command line messages are designed to be self-explanatory. If you have trouble understanding a message, refer to the "Troubleshooting" section of the OpenDDA Reference Manual.

Example of OpenDDA command line messages

The following is an example of the OpenDDA command line messages you will see when you invoke the Complete Build function. This example shows the OpenDDA tasks performed:

- parsing the EDB
- expansion of the execution statements

```
dda -cb -o -Lc ddademo.c
Begin processing EDB...
DDA-S, EDB processing completed successfully.

Begin validating external data...

Processing Statistics for:
  ddademo.c

Total EDB Data Definitions Parsed:      3
Total Dynamic Access Records Built:     0
Total Static Access Records Built:      3

Total Records Processed by Platform(s): 3

Total Errors Detected:                   0
Maximum Errors Allowed:                  20

Total Items designated for Input:        3
Total Items designated for Output:       3
DDA-S, External data validated successfully.

Expanding execution statements...

Read 219 lines, found 0 errors and 0 warnings.
Wrote 467 output lines.
DDA-S, Execution statements expanded successfully.
```

Redirecting command line messages to a file

If you need to redirect the output to a file, use the standard mechanism that the shell you are using provides.

Example:

In csh (C shell), you would enter the following:

```
dda -cb -o filename.f > out.txt
```

5.5 OpenDDA Completion Status

OpenDDA completion status

Whenever an OpenDDA function is invoked from the command line interface, a completion status is returned.

This completion status will indicate “no error” whenever the function was entirely successful. Whenever any error of any kind is encountered, the completion status will indicate “error.”

OpenDDA completion status values

A value is placed in the status environment variable to indicate whether an error occurred. The following table illustrates the potential values of status:

	No Error	Error
status	0	non-zero

Using the OpenDDA completion status

The following is an example of how you can use the OpenDDA completion status in a C shell script:

```
dda -cb -o myapp.f
if ($status != 0) then
    goto cb_error
endif
# complete build was successful, so compile & link myapp
comp&link myapp
exit
cb_error:
echo "Error encountered during Complete Build"
exit
```

5.6 OpenDDA Command Line Help

Obtaining Help from the command line interface

To obtain help from the command line interface, simply type

dda -h

This will print help information to Standard Output.

5.7 Using OpenDDA Functions from Script Files

Using host platform script files

Any OpenDDA command line can be put into a Host Platform script file and executed by executing the script file. This allows you to automate OpenDDA application development by combining frequently used combinations of OpenDDA functions in the order you prefer.

You can even use your Host Platform's ability to accept parameters to make flexible script files.

Script file example

The following C shell example script illustrates the use of OpenDDA commands in a script file.

```
# builds the ddademo.f application
#
/opt/DDA/bin/dda -dv ddademo.def
if ($status != 0) then
    echo "\nError during ddademo.def file validation"
    exit
endif

/opt/DDA/bin/dda -cb -o -Lf ddademo.f
if ($status != 0) then
    echo "\nError during complete build of ddademo.f"
    exit
endif

/opt/DDA/examples/dda_create_fortran_executable ddademo
if ($status != 0) then
    echo "\nError during compile/link of ddademo"
endif

exit
```

5.8 Using OpenDDA Functions in a Program

Introduction

This section describes what to do if you have a desire to perform any OpenDDA functions in a program. For example you may:

- re-build an existing OpenDDA application
- modify an existing OpenDDA application
- build a new OpenDDA application
- create a CIO file

The SYSTEM call

Many FORTRAN and C compilers support a “SYSTEM” call. A system call allows you to execute any HP-UX command from within a program. Use the system call to execute an OpenDDA function. Refer to the documentation provided by your compiler for detailed information on how to accomplish this.

Examples

Fortran Example:

```
system("/opt/DDA/bin/dda -cb -o -Lf myapp.f")
```

C Example:

```
system("/opt/DDA/bin/dda -cb -o -Lc myapp.c");
```

5.9 Parse External Data Block

Purpose

The purpose of the Parse External Data Block function is to

- check the External Data Block (EDB) for proper syntax
- if the syntax is correct, validate the external data defined in the EDB.

Also, for external data sources that support the notion of a handle that optimizes data access, the handles will be obtained at this time.

Command line qualifier

Use the following qualifier to invoke this function from the command line: `-pe`

Input file

A file containing an OpenDDA External Data Block is required input to this function. You need to provide the name of this file. It can have any file name you wish.

`-pe myfile.f`

Output file

The successful result of this function creates the following files:

- OpenDDA Set Definition file
 - A file containing EDB set definition information that will be used by other OpenDDA functions during application development. It has the same name as the input filename with a `.set` file extension.
- OpenDDA Set Index file
 - A file containing EDB set definition information that will be used by other OpenDDA functions during application development. It has the same name as the input filename with a `.set_idx` file extension.
- OpenDDA Control Input/Output file
 - A file containing EDB input and output information required by your OpenDDA application when executing. This file has the same name as the input filename with a `.cio` file extension.

Valid options

The following options can be used with this function:

- X Use External Data – overrides whatever is specified in the External Data Block.
- T Use Test Data – overrides whatever is specified in the External Data Block.
- S Parse External Data Block syntax only, no output files are generated.
- LC or -Lf Language specifier. The -LC option specifies C and is required if your source code is in C. The -Lf option specifies FORTRAN. FORTRAN is the default and therefore -Lf is optional if the source language is FORTRAN.
- O Overwrite Existing Output Files

Remote node platform restrictions

There are two things to remember about remote node platforms:

1. You cannot validate external data from a remote node; therefore, this command will only work on the local Application Module^x unless:
 - the /TEST_VALUE qualifier is on every EDB statement, or
 - you have used the -T option (Use Test Data).
2. If an OpenDDA application is being developed from a remote node, after the application is copied to the Application Module^x, the EDB must be re-parsed to access external data.

Examples

This table contains examples of using this function from the OpenDDA Command Line Interface:

This command...	Causes this to happen... (Continued)
<code>dda -pe -X -o myfile.edb</code>	The file <code>myfile.edb</code> will be checked for proper syntax. If the syntax is correct, the external data will be validated and these files will be written (or overwritten if they already exist): <code>myfile.set</code> <code>myfile.set_idx</code> <code>myfile.cio</code>
<code>dda -pe -T -o newfile</code>	The file <code>newfile</code> will be checked for proper syntax. If the syntax is correct, these files will be written (or overwritten if they already exist) using the test values specified in the EDB: <code>newfile.set</code> <code>newfile.set_idx</code> <code>newfile.cio</code>
<code>dda -pe -X thisfile.003</code>	The file <code>thisfile.003</code> will be checked for proper syntax. Even if the syntax is correct and the external data is validated, the files: <code>thisfile.set</code> <code>thisfile.set_idx</code> <code>thisfile.cio</code> will NOT be written since the <code>-O</code> option is not specified—unless 1) The files do not already exist—in which case they will be created or 2) You respond "yes" to the prompt asking you if you want the files overwritten.

5.10 Expand EXEC DDA Statements

Purpose

The purpose of the Expand EXEC DDA Statement function is to

- check the EXEC DDA statements for proper syntax
- expand EXEC DDA statements into OpenDDA specific functions that will accomplish the semantics specified by the EXEC DDA statements and insert those OpenDDA specific functions into the application source code.

The successful result of the Expand EXEC DDA statements function creates a source file that is ready to be compiled and subsequently used to create an executable image (refer to "Output file").

Command line qualifier

Use the following qualifier to invoke this function from the command line:

-ee

Input file

The following files are required input to this function. Not all of them are explicitly provided by *you*:

Explicit input file

- Application Source Code with embedded EXEC DDA statements.
You need to explicitly provide the filespec for this file. It can have any filespec you want.

Implicit input file

- OpenDDA Set Definition file
You only provide the filespec for this file when using the “-set” option. Otherwise, OpenDDA will implicitly expect a filespec with the following properties:
 - the same filename as the Application Source code file
 - a “.set” file extension
 - located in the same directory as the Application Source Code
 - created by the Parse External Data Block function.

- **OpenDDA Set Index file**
 You do not need to provide the filespec of this file. OpenDDA will implicitly expect a filespec with the following properties:
 - same filename as the Set file
 - a “.set_idx” file extension
 - located in the same directory as the Application Source Code
 - created by the Parse External Data Block function.

Output file

The following file is created with a successful result of this function:

- **OpenDDA Application Source Code**
 with OpenDDA functions replacing the EXEC DDA statements.

This file will have the same filename as the application source code with the file extension `_src.f` or `_src.c` appended to it.

Valid options

Any of the following options can be used with this function:

- o Overwrite Existing Output Files
- LC or -Lf Language specifier. The -LC option specifies C and is required if your source code is in C. The -Lf option specifies FORTRAN. FORTRAN is the default and therefore -Lf is optional if the source language is FORTRAN.
- set *filename.set* Expand EXEC DDA Statements that relate to an External Data

Block contained in a file other than the file containing the EXEC DDA Statements.

Note: The -set option must be followed by the name of the ".set" file created by the Parse External Data Block function.

Examples

The following are examples of using this function from the OpenDDA Command Line Interface.

This command...	Causes this to happen...
<code>dda -ee -o myfile.f</code>	The file <code>myfile.f</code> will be checked for proper EXEC DDA syntax. If the syntax is correct, the EXEC DDA statements will be expanded into OpenDDA specific functions and the following file will be written (or overwritten if it already exists): <code>myfile_src.f</code> This function assumes the following files exist in the same directory as <code>myfile.f</code>: <code>myfile.set</code> <code>myfile.set_idx</code>
<code>dda -ee -set special.set normal.f</code>	The file <code>normal.f</code> will be checked for proper syntax. Even if the syntax is correct, the following file: <code>normal_src.f</code> will NOT be written since the <code>-o</code> option is not specified—unless: 1) The file does not already exist—in which case it will be created or 2) You respond "yes" to the prompt asking you if you want the file overwritten. This function assumes the following files exist in the same directory as <code>normal.f</code>: <code>special.set</code> <code>special.set_idx</code>

5.11 Complete Build

Purpose

The purpose of the Complete Build function is to perform the following two OpenDDA functions:

- Parse External Data Block (-pe)
- if the Parse External Data Block succeeds, Expand EXEC DDA Statements (-ee).

Refer to 5.9 and 5.10, respectively, for more information on these OpenDDA functions.

Command line qualifier

Use the following qualifier to invoke this function from the command line:

-cb

Input file

This function requires the following file:

- Application Source Code with embedded EDB and EXEC DDA statements. You need to explicitly provide the filespec for this file. It can have any filespec you want.

Output file

The following files are created as a result of this function:

- OpenDDA Set Definition file
 - A file containing EDB set definition information that will be used by other OpenDDA functions during application development. It has the same name as the application source code file with a .set file extension.
- OpenDDA Set Index file
 - A file containing EDB set definition information that will be used by other OpenDDA functions during application development. It has the same name as the application source code file with a .set_idx file extension.
- OpenDDA Control Input/Output file
 - A file containing EDB input and output information required by your OpenDDA application when executing. This file has the same name as the application source code file with a .cio file extension.
- Application Source Code file
 - A file with OpenDDA functions replacing the EXEC DDA statements. This file has the same filename as the application source code, with a _src.f or _src.c file extension.

Valid options

The following options can be used with this function:

- X Use External Data – overrides whatever is specified in the External Data Block.
- T Use Test Data – overrides whatever is specified in the External Data Block.
- LC or -Lf Language specifier. The -LC option specifies C and is required if your source code is in C. The -Lf option specifies FORTRAN. FORTRAN is the default and therefore -Lf is optional if the source language is FORTRAN.
- O Overwrite Existing Output Files

Example

The following example uses this function from the OpenDDA Command Line Interface.

This command...	Causes this to happen...
dda -cb -o myfile.f	The file <code>myfile.f</code> will be checked for proper EXEC DDA syntax. If the syntax is correct, the EXEC DDA statements will be expanded into OpenDDA specific functions and the following file will be written (or overwritten if it already exists): <code>myfile_src.f</code> OpenDDA will also write these files in the same directory as <code>myfile.f</code>. Those files are <code>myfile.set</code> <code>myfile.set_idx</code> <code>myfile.cio</code>

5.12 Validate Definition File Syntax

Purpose

This function validates the syntax of the options specified in the application's Definition file. This should be used anytime you modify your Definition file to ensure that when your OpenDDA application executes it will be able to read the contents its Definition file.

Command line qualifier

Use the following qualifier to invoke this function from the command line:

-dv

Input file

This function requires the following file:

- OpenDDA Definition file

You need to provide the name of this file. The filename must be the same as the application's filename. The file extension must be .def.

If you do not explicitly include the .def file extension, OpenDDA will append it to the filename you provide.

Output file

None

Valid options

None

Example

The following is an example of using this function from the OpenDDA Command Line Interface.

This command...	Causes this to happen...
dda -dv myfile	The file <code>myfile.def</code> will be checked for proper syntax. The <code>.def</code> file extension was not explicitly provided, it is the default. • If the syntax is correct, a successful message is printed to Standard Output. • If the syntax is incorrect, errors are printed to Standard Output.

Directives that are no longer required

The following .def file directives were required in OpenDDA Releases 100 and 110, but are not used in Release 200 or 300:

- NAME
- DESCRIPTION
- TYPE

In OpenDDA Release 200 and 300, these directives are allowed for backward compatibility, but are not used. If present, however, they will be checked for syntax. If you validate a .def file that includes these directives, advisory messages will be printed indicating that the directives are not needed and can be removed.

DEF file optional

Use of the definition file is optional.

5.13 Install Application

Application Directory

The executable file for your application resides in the Application Directory. The files that support your application (the CIO file and optional DEF file) also reside in this directory. The application source file can also reside here, but this is not required. The Application Directory, by definition, is the directory from which application installation is done.

Purpose of Install

This function performs several tasks to prepare your OpenDDA application to be executed online including creating a symbolic link to the OpenDDA application's Executable Image located in the Application Directory. OpenDDA also examines the pathname of this link to locate the CIO file and optional DEF file.

This link will be created in the controlled AxM directory (/users/axm) designated as the location for the execution of AxM CL applications.

File protection

The following table shows which file protections are set for the application executable file when an install is performed.

Table 5-5 File Protections

File	Level	Protection
my_application	World	Read, Write, Execute=OFF
	Group	Execute=ON

Command Line Qualifier

Use the following qualifier to invoke this function from the command line:

-is

Input file

The following file is required input to this function

- The Executable Image

Output file

None

Valid options

None

Example

The following is an example of using this function from the OpenDDA Command Line Interface.

This command...	Causes this to happen...
<code>dda -is myapp</code>	A symbolic link to myapp will be created in the A ^x M directory designated as the location for A ^x M CL (<code>/users/axm</code>) applications to execute from. The A ^x M Group Level execution protection for myapp (in the Application Directory) will be turned on..

After installation

Once the install command is complete, a CL program must be written on the LCN. This CL program must contain the appropriate subroutine call to execute an "x-side" application. Below is an example of a generic CL program that will perform this function.

```
-- AXM EXECUTE TASK WITH WAIT TEST
--
PACKAGE
--AXM test CDS STARTCDS
--
CUSTOM (ACCESS OPERATOR)
--Application to be started
PARAMETER STRTAPPL      : STRING
--
--Return status from the AMCL06$EXECUTE_TASK_WITH_WAIT call
PARAMETER RET_STAT      : NUMBER
NOT BLD_VISIBLE
--
--Return detail status from AMCL06$EXECUTE_TASK_WITH_WAIT call
PARAMETER DET_STAT      : NUMBER
NOT BLD_VISIBLE
--Timeout used in the AMCL06$EXECUTE_TASK_WITH_WAIT call
PARAMETER X_TMOUT       : TIME
--Timeout to use in above function call
PARAMETER REQTMOUT      :TIME
END CUSTOM
BLOCK STARTX (GENERIC; AT BACKGRND)
--
%INCLUDE_SET AMCL06
CALL AMCL06$EXECUTE_TASK_WITH_WAIT(RET_STAT,
&                                     DET_STAT,
&                                     STRTAPPL,
&                                     X_TMOUT,
&                                     REQTMOUT)
END STARTX
END PACKAGE
```

After the CL is compiled, attach it to the scheduled point you want to execute your application. The example provided uses a generic parameter called STRTAPPL. Modify this string to the name of your installed application in the /users/axm directory.

ATTENTION

If your OpenDDA application is executed from a Universal Station operator keyboard, which has only **UPPERCASE** output, your CL program must be written to acknowledge this case-sensitivity issue for your OpenDDA application name.

Completion status

There are two status parameters populated when the application executes:

- RET_STAT, and
- DET_STAT.

A successful execution and completion of the application will return a zero to both of these fields. Refer to the CL/AM Reference Manual for more information on the definition of these return values.

ATTENTION

If the DET_STAT field contains a value not found in the CL/AM Reference Manual, then the value returned was the exit status of your application.

5.14 De-Install Application

Purpose

This function performs several tasks to prevent your OpenDDA application from being executed online. These tasks include

- turning off the execution privilege of the application, and
- removing the symbolic link to the OpenDDA application's Executable Image

Command line qualifier

Use the following qualifier to invoke this function from the command line:

-ds

Input file

This function requires the following file:

- the Executable Image

If this file is not found, the symbolic link will still be removed.

Output file

None

Valid options

None

Example

The following is an example of using this function from the OpenDDA Command Line Interface.

This command...	Causes this to happen...
dda -ds myapp	A symbolic link to the executable file myapp will be deleted. The symbolic link to the executable file myapp will be removed from the A^xM directory designated as the execution location for A^xM CL (/users/axm) applications.

6. File Management

6.1 Overview

Introduction

OpenDDA applications include several related files.

This section describes the various files required to form an OpenDDA application so you can manage them effectively.

ATTENTION

The files listed in the following table are in the order they are normally created during development of an OpenDDA Application.

6.2 OpenDDA Files

OpenDDA file types

The following table lists information about all the files related to an OpenDDA Application, including

- the name of each file (case sensitive),
- the file extension required of the file (case sensitive),
- a description, and
- how they are created.

Table 6-1 List of OpenDDA Application File Types

Name of File	File Extension	Description	Created by...
FORTRAN Source Code	f	FORTRAN source code containing EXEC DDA statements. The External Data Block may be included in this file or may be in a separate file.	OpenDDA application developer
C/C++ Source Code	c (C) c or C (C++)	C/C++ source code containing EXEC DDA statements. The External Data Block may be included in this file or may be in a separate file.	OpenDDA application developer
External Data Block (EDB)	<i>anything or nothing</i>	A file containing an External Data Block for an OpenDDA application.	OpenDDA application developer
Definition (DEF)	def	A file containing OpenDDA application directives which tell the executing application how to behave. This file is used while executing an OpenDDA application.	OpenDDA application developer
Set Definition	set	A file containing information relating to the Sets defined in the External Data Block. This is an OpenDDA specific file that is needed only during development of an OpenDDA application.	OpenDDA Parse Function
Set Index	set_idx	A file containing information relating to the Sets defined in the External Data Block. This is an OpenDDA specific file that is needed only during development of an OpenDDA application.	OpenDDA Parse Function

Table 6-1 List of OpenDDA Application File Types

Name of File	File Extension	Description	Created by...
Control Input/ Output (CIO)	cio	A file containing information relating to the external data to be input and output by the OpenDDA application. This file is used while executing an OpenDDA application.	OpenDDA Parse Function
Expanded Application Source	_src.f	The original FORTRAN Application Source Code with the EXEC DDA statements expanded into OpenDDA Function calls.	OpenDDA Expand Execution Function
Expanded Application Source	_src.c	The original C/C++ Application Source Code with the EXEC DDA statements expanded into OpenDDA Function calls.	OpenDDA Expand Execution Function
Object	_src.o	The Expanded Application file compiled into object form.	Compiler
Executable Image	<i>anything</i>	The OpenDDA application in a format capable of being executed on your Host Platform. This file is necessary to execute the OpenDDA application.	Linker
General Output	dda_out	This file contains any output directed to Standard Output. This includes any output specified by the DEF file.	An executing OpenDDA Application
Include files	h (C) f (FORTRAN)	These files contain OpenDDA specific definitions and errors	OpenDDA Installation

7. Using OpenDDA Statuses

7.1 Overview

Introduction

OpenDDA Execution Statements always have information to return to you concerning the results of the operation. This information is returned in your HLL Status Variables. This section explains

- terminology,
- how to interpret an OpenDDA Status, and
- how to determine corrective action.

OpenDDA Status Value defined

OpenDDA Status Values are values returned to an application by an OpenDDA Execution Statement. These values are used to indicate various degrees of success or failure. They are used with the following variables:

- Overall Execution Status
- Value Status
- Store Status
- External Reference Status

OpenDDA Status Mnemonic defined

The numerical representations of OpenDDA Status Values are subject to change from release to release. Therefore, each OpenDDA Status Value has a mnemonic to represent the actual value. You should never deal directly with the numerical representation in an OpenDDA application. You should always use an OpenDDA Mnemonic. The meaning associated with an OpenDDA Mnemonic will not to change from one release of OpenDDA to the next. The mnemonics are used exclusively throughout the OpenDDA user documentation whenever a reference is made to a specific OpenDDA Status Value. The mnemonics are known to an OpenDDA Application by including the HLL include file (see the *OpenDDA Reference Manual*, subsection 6.3).

OpenDDA Status Variables

There are two types of statuses returned by an OpenDDA Execution Statement:

- **Overall**—reports the overall results of an Execution Statement.
- **Individual**—reports the results of the Execution Statement for each member of the Set being acted on. The External Data Block designates certain HLL Variables that will receive OpenDDA Status Values as the result of an Execution Statement.

The following table lists these HLL Variables and identifies which Execution Statement causes them to return the results of the operation:

Table 7-1 OpenDDA Status Variables

HLL Variable	Status Type	Execution Statement
Execution Status Variable	Overall	all Execution Statements
Value Status Variable	Individual	EXEC DDA READ
Store Status Variable	Individual	EXEC DDA WRITE
External Reference Status Variable	Individual	EXEC DDA MODIFY EXEC DDA LOAD ALIAS

7.2 Interpreting OpenDDA Status Values

Introduction

An OpenDDA Status Value is composed of several pieces of information. This subsection explains the composition of an OpenDDA Status Value by breaking it down into its component parts. By understanding the composition you will know how to evaluate an OpenDDA Status Value.

Status Components

OpenDDA Status Values are composed of three parts. They are:

- Facility Code
- Message Number
- Severity

Facility Code

The Facility Code indicates which software component is reporting the status. The Facilities are:

Table 7-2 Facility Codes

Facility Code	Facility Code Mnemonic	Reported by...
HGSC	HGSC_FACCODE	Generic statuses that can be returned from many DCSs
DDA	DDA_FACCODE	OpenDDA functions
GDX	GDX_FACCODE	Generic Data eXchange system
LCN*	LCN_FACCODE LCN_DMINOR_FACCODE LCN_PARAMACCESS_FACCODE LCN_AXE_FACCODE	TDC3000 DCS
SYS	SYS_FACCODE	HP-UX system

* Because of the large potential range for the Message Numbers from the LCN, the LCN statuses are split across four Facilities.

Message Number

The Message Number is a unique number within each Facility that identifies a specific status. For some of the Facilities, the Message Number contains additional information that may be of interest to the application. The following table identifies the contents of the Message Number for each Facility.

Table 7-3 Meaning of Message Number

Facility	Message Number Is...
HGSC	An integer assigned by the HGSC Facility
DDA	An integer assigned by the DDA Facility
GDX	An integer assigned by the GDX Facility
SYS	The HP-UX 'errno' value
LCN	An integer assigned by the LCN Facility

Severity

The severity indicates how serious the status is. Note that Successful and Informational messages are odd, and Warning, Error, and Fatal messages are even. The severities and their numerical values are:

Table 7-4 Severity Values

Severity	Value	Meaning
Success	1	The operation was successful.
Informational	3	The operation was successful. The complete OpenDDA Status Value also provides additional information about the processing that took place.
Warning	0	The operation did not fully complete. Partial processing may have been performed. The complete OpenDDA Status Value also provides additional information about the processing that took place.
Error	2	The operation failed completely. The complete OpenDDA Status Value also provides additional information about the processing that took place.
Fatal	4	The operation encountered an error that prevented processing from continuing. The complete OpenDDA Status Value also provides additional information about the processing that took place.

Extracting the status components in a C application

The structure definitions provided in `dda_status_include.h` (located in `/opt/dda/include`) allow a C application to extract the components from the status value.

- `STATUS_CODE_UNION` - provides a union of the status as a long integer (as returned by OpenDDA) and as a structure of `STATUS_CODE_TYPE`.
- `STATUS_CODE_TYPE` - allows direct access to the components of the status.

Example

This example shows how to extract the components of an OpenDDA status in a C application.

```
#include "dda_status_include.h"

/* Variables */
DDA_STATUS_TYPE      Exec_Status;
STATUS_CODE_UNION    Status_Union;
long                 Facility;
long                 Message_Number;

EXEC DDA READ SET='ALL', STATUS=Exec_Status;

if ( (Exec_Status & 1 ) == 1) /* Odd Severity is OK */
{
    /* Success or Informational, continue ok */
    ...
}
else /* Extract the Facility and Message Number */
{
    Status_Union    = Exec_Status;
    Facility        = Status_Union.FacilityCode;
    Message_Number  = Status_Union.MessageNumber;

    /* See if the error is from the system */
    if (Facility == SYS_FACCODE)
    {
        /* print the HP-UX errno value */
        printf("errno is %d\n", Message_Number);
    }
    else
    {
        /* some other error */
        ...
    }
}
```

Extracting the status components in a FORTRAN application

FORTRAN applications can access the status components by extracting them using the MVBITS intrinsic function. The MVBITS subroutine is called as follows:

```
call MVBITS ( source, source_start_bit, size, destination,  
             destination_start_bit )
```

which means to move *size* bits from *source* starting at bit *source_start_bit*, to *destination* starting at bit *destination_start_bit*.

The following table identifies where each component is in the status:

Table 7-5 Location of Status Components

Component	Start Bit Position	Size in Bits
Facility Code	16	16
Message Number	3	13
Severity	0	3

For more information on MVBITS, refer to the *HP FORTRAN/9000 Programmer's Reference* manual.

Example

This example shows how to extract the components of an OpenDDA status in a FORTRAN application.

```
C Variables
integer*4 exec_status, facility, message_number, severity

EXEC DDA READ SET='ALL', STATUS=exec_status;

call MVBITS( exec_status, 16, 16, facility, 0 )
call MVBITS( exec_status, 3, 13, message_number, 0 )

IF ( ODD(exec_status) ) THEN
C   Success or Informational, continue ok
    ...
ELSE
C   Extract the Facility and Message Number

    call MVBITS( exec_status, 16, 16, facility, 0)
    call MVBITS( exec_status, 3, 13, message_number, 0)

C   See if the error is from the system

    IF ( facility .EQ. SYS_FACCODE ) THEN

C       Print the HP-UX errno value

        PRINT *, 'errno is: ', message_number
    ELSE

C       Some other error
        ...
    ENDIF
ENDIF
```

7.3 Determining Corrective Action

Introduction

Depending upon the information returned by an OpenDDA Status Value, there may or may not be corrective action you can take. This section explains how to determine the corrective action to take based upon an OpenDDA Status Value.

Using the Severity component

The Severity status component can be used as an indicator of what action should be taken. The following table illustrates what to do for each Severity. See the next topics for information about how to determine the status mnemonics.

Table 7-6 Actions for each Severity Level

Severity	Action
Success	The operation was successful. No corrective action is necessary.
Informational	The operation was successful. You may want to check the status value(s) against the status mnemonics to see what other information is being communicated.
Warning	The operation did not fully complete. Check the status value(s) against the status mnemonics to determine if processing should continue.
Error, Fatal	The operation failed. You may want to check the status value(s) against the status mnemonics to determine why the operation failed.

Obtaining OpenDDA Error Mnemonic

If your OpenDDA application receives an unexpected status and you wish to obtain the status mnemonic, use the following script.

```
if[$# -ne 1]
then
    echo "\nUsage: finderr <errnumber>"
    echo "      where <errnumber> must be a decimal value\n"
else
    awk '
        BEGIN{
            search="0x0" toupper(sprintf("%7x","$1"));
            gsub("","0",search);
        }
        $0 ~ search {print $2}
    ' /opt/DDA/include/dda_c_errors.h /opt/DDA/include/ hgsc_c_errors.h
    /opt/DDA/bin/dda_xstatus $1
fi
```

Note: This script is NOT provided online.

You will have to write the script file, including the commands that are given above. This script should be written as a HP-UX script in a file called **finderr**. Change the mode of the script file **finderr** to be executable by using the following command:

```
chmod 777 finderr
```

Then, you can execute it from the HP-UX shell. You will be required to provide one argument—the decimal value of the error code.

Example

The following is an example of how to use the script:

```
finderr 244023426
```

The tool will return the following output:

```
DDA_INITIALRDYCALLED  
DDA-E, INITIALIZE has already been called
```

The mnemonic to be used in the application is DDA_INITIALRDYCALLED.

Corrective action

After the OpenDDA Status Mnemonic has been determined using the `finderr` script shown above, look up the OpenDDA Execution Statement that returned the status in the OpenDDA Reference Manual, Section 6. The tables describing the status values returned will indicate what action should be taken to correct the problem.

8. Remapping CIO Files

8.1 Overview

Introduction

Many OpenDDA applications will access a fixed set of external data. That is, each time that external data is transferred to or from a specific native language variable, the transfer will always be to or from the same external data entity. There will be applications, however, where it is desirable to change, at run time, the external data that is assigned to native language variables. For example, the application could read A100.PV into native language variable *var1* some of the time, and read B100.PV into *var1* at other times.

For each OpenDDA variable, there is a corresponding native language variable. The relationship of OpenDDA variables to external data is defined in the Control Input/Output (CIO) file.

What is a CIO file?

In the most basic form of an OpenDDA application, the application source includes FORTRAN or C source code, and an OpenDDA External Data Block (EDB) that can be included in the source code file or can be separate. The EDB contains statements that define OpenDDA variables for the external data that the application will access (read and/or write). The application source code includes FORTRAN or C variables that correspond directly to the external data variables defined in the EDB. The principle function of OpenDDA is to read/write the external data to/from the application high-level language variables.

When OpenDDA parses the EDB, it validates the external data and then generates the Control Input Output (CIO) file. The CIO file contains a mapping of the EDB value variable to the physical address of the external data as it is known to the platform. The following shows the hierarchical mapping of data from the application to the platform internal form:

<u>APPLICATION SOURCE</u>	<u>EDB</u>	<u>CIO FILE</u>
---------------------------	------------	-----------------

FORTTRAN/C Variable -->	EDB ValueVariable -->	Platform Address
-------------------------	-----------------------	------------------

In this basic form of the OpenDDA application, the external data locations specified in the EDB cannot be changed without editing and reparsing the EDB.

Dynamically changing external data

OpenDDA provides two techniques that allow applications to switch external data at runtime. These two techniques are:

- Remapping CIO files
- Dynamic Selection Sets

An application can use either technique as appropriate, or can use features from both for additional flexibility. This Section will cover remapping CIO files. The next will cover dynamic selection sets.

8.2 Remapping CIO Functionality

The remapping concept

Beginning with Release R200, OpenDDA provides the user with the tools to dynamically remap the application variables to a different set or sets of external data by allowing an application to read multiple CIO files into memory and to select the one that will be active at any given time. This is accomplished by defining a separate EDB for each set, or "instance" of external data. The EDB files are then individually parsed to generate a separate CIO file for each EDB. OpenDDA EXEC DDA functions allow the application to read, write and clear CIO files to/from memory, and select the one to be active. It is then possible to use the different CIOs to read/write data to the platform using the same common high level language variables.

The remapping tools

Four OpenDDA EXEC DDA statements are provided to support remapping.

- EXEC DDA READ CIO—reads a specified CIO file from disk into memory
- EXEC DDA WRITE CIO—writes a specified CIO file from memory to disk
- EXEC DDA SELECT CIO—selects a specified memory-resident CIO file to be the active one
- EXEC DDA CLEAR CIO—clears a specified memory-resident CIO file from memory

These statements are described in detail in Section 6 of the *OpenDDA Reference Manual*.

Maximum number of CIOs

The maximum number of CIOs that can be read in and stored in memory is limited only by the amount of system memory available.

Similarity to Equipment List

There are many possible uses for the remapping capability. For example, one could use a single application program to control or collect data from a number of physically different but functionally similar equipments (unit instances). Three equipments could be making variations of the same product. Each equipment could have its own EDB defining the specific points and parameters for that equipment. The Remap CIO functionality, when so applied, is similar to the functionality of the TDC 3000^x Equipment List, except that it provides this capability at the HP-UX level. See the *Equipment List Reference Manual* for information about Equipment List functionality and the concept of unit instances.

One difference to note is that overloading of high level language variables is allowed with OpenDDA. This capability provides the user with the ability to use common high level language variable names across multiple sets. This could support a "sub" unit level of granularity to support items like primary/secondary pumps, multiple sensors, and other variable External References that may be specified based on different operational scenarios.

The EDBs must be structurally identical

The variables in an EDB correspond directly to FORTRAN or C variables in the application. Therefore, if an application has multiple EDBs, they must all contain the same number of variables, and they must be of the same type and in the same order so that they will all map properly to the high-level language variables.

In our simple example, if one of the units did not have a particular external data element that was present in the other two units, the NOX option could be used to pad the "short" EDB to make it identical in form to the other two EDBs. The NOX variables could be changed to valid static references at a later time, but could **not** be changed to dynamic references.

The CIO signature

OpenDDA provides a mechanism to ensure that all EDBs for an application are of the same form to prevent undetected errors at run time. That mechanism is a special checksum called the CIO Signature. The CIO Signature is generated when the EDB is parsed, and is included in both the CIO file and in the application. When a CIO file is read into memory by an application, the signature of the CIO file is checked against the signature embedded in the application. This check can be enabled or disabled by a DEF file directive named CIO_SIGNATURE. If the check is enabled and the signatures do not match, an error status is returned to the application. This then prevents any subsequent actions against this specific CIO.

How EDBs can differ

The following is a list of the only items allowed to be different between two EDBs and still generate the same CIO signature:

- EXTDEF Statements
- Access Name
- Non-NULL Point.Parameter (index) external data location specification
- Test Values along with the use of /TEST_VALUE and /NOTEST_VALUE
- The order of External Data Statement Components within an External Data Statement
- Implicit defaults vs. explicit specification of a default (For example, Data Type derived from Test Value or explicitly specified by means of a qualifier
- The case of FORTRAN variable names (for example, VAR1 is the same as var1). Note: Since the C and C++ languages are case sensitive, this does not apply for them.
- The order of SET_LIST elements (for example, SET_LIST=A, B, C is the same as SET_LIST = B, A, C)
- Adding or deleting an external data array index specification to/from an existing External Data Statement as long as it remains a scalar element; for example, FIC101.PV can change to AMPOINT.CDSREAL(1)
- CURRENT vs. NOX Location Specifier

How EDBs cannot differ

The following items cannot be different. If any of these items are different, the CIO signatures will be different.

- The number of External Data Statements
- The order of the External Data Statements (the order of components within an External Data Statement can be different)
- The data types of HLL variables
- The names of HLL variables
- The case of C/C++ variable names (FORTRAN is not case sensitive)
- Adding or deleting any ancillary variables in an External Data Statement
- Adding, deleting, or changing the names of the SET_LIST elements

9. Dynamic Selection Sets

9.1 Overview

What will be covered

This section will give an overview of the Dynamic Selection Sets functionality. There are a number of pieces that comprise this functionality—EDB statements, EXEC DDA statements, and the selection sets themselves. This section is intended to cover the concepts, giving an overall view of the process of dynamic data selection. The details are covered in the *OpenDDA Reference Manual*.

Introduction

Dynamic Selection Sets functionality allows an application to modify one or more External Data References within the currently active memory resident CIO. This functionality provides, at the HP-UX level, a capability similar to Dynamic Indirection that is available, at the control platform level, in TDC 3000^x in the Application Module. Both Dynamic Selection Sets and Dynamic Indirection allow a user to switch data references at run time. For more information about Dynamic Indirection, see subsection 2.3.8 of the *CL/AM Reference Manual*.

Comparison with Remap CIO

The OpenDDA Remap CIO functionality provides the user with the tools to dynamically remap the application variables to a different set or sets of external data by allowing an application to read multiple CIO files into memory and to select which one will be active at any given time. (See “Remapping CIO Files”.) This has the effect of “batch” switching of External References with a single OpenDDA EXEC statement (EXEC DDA SELECT CIO).

With the OpenDDA Dynamic Selection Sets functionality, the user can modify the External Reference **within** the memory resident version of the currently active CIO. External References can be modified at the point, parameter, or index level, or combinations thereof. In addition, External References can be grouped and modified in sets—Dynamic Selection Sets—providing considerable flexibility.

It is also possible to have multiple references dynamically changed with the specification of a single External Reference component. This also provides capability similar to dynamic indirection where dynamically modifying one point name component can impact multiple separate references.

The tools for Dynamic Selection Sets

OpenDDA provides the following tools, and related syntax, to achieve the dynamic selection of sets of External Data References.

- Dynamic Selection Sets—specify sets of External References that will be modified as a group
- EDB support
 - Alias Variables—variables that are loaded by the application with desired External Reference component names prior to a MODIFY action (components are point, parameter, and index)
 - External Reference Status Variables—return, for each External Reference to be modified, the status of the MODIFY operation (optional, but highly recommended)
 - Modify Code Variables—allow the application to select, at run time, whether or not an individual reference will be modified (optional)
- EXEC DDA statements
 - EXEC DDA MODIFY CIO—Modifies External References in the CIO by updating them with names contained in Alias Variables
 - EXEC DDA LOAD ALIAS—Loads Alias Variables with values from the currently active CIO
 - EXEC DDA LOAD MODIFY_CODE—Batch loads all Modify Code variables with a specified value

Location Specifications that are valid

The following table shows Location Specifications that are valid to use with Dynamic Selection functionality.

Table 9-1 Valid Location Specifications for Dynamic Selection

Location Specification	Valid
CURRENT	Yes
NOX	No

Terminology

The following terminology is used in this discussion of Dynamic Selection Sets.

- **External Reference**—shorthand notation for External Data Reference. An External Reference consists of:
 - a **name**—identifies the External Reference—for example, point1.param6(1)
 - a **status**—determines if the External Reference represents a valid external data item
- **Dynamic Selection Sets**—sets that exist to define:
 - which External Data Statement's External References will be modified
 - which Alias Variables will specify the new External Reference
- **Alias Variable**—the variable that your application will load with an ASCII representation of an External Reference
- **Dynamic Selection**—the process by which External References are selected at run time

9.2 OpenDDA Sets

Two types of sets

OpenDDA uses sets to define which External Data Statements are acted upon by certain EXEC DDA statements. There are two types of sets:

- Dynamic Selection Sets define the variables whose external references will be modified by an EXEC DDA MODIFY CIO.
- Data Access Sets define the data that will be read or written by an EXEC DDA READ/WRITE.

How variables are assigned to sets

Variables are assigned to sets with the SET_LIST directive in the EDB. The SET_LIST directive automatically assigns the associated Value Variable to the specified Data Access Set(s). In addition, if the External Reference has dynamic components specified by Alias Variables, those Alias Variables will also be included in Dynamic Selection Set(s) that share the same names with Data Access Sets.

The following External Data Statement contains an Alias Variable. The syntax for specifying External Reference components and corresponding Alias Variables will be covered later in this section. At this point, for the purpose of illustrating assignment of variables to sets, accept that Var1Point_Name is an Alias Variable representing an Entity Id on the system.

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.PV)
Var1Point_Name = POINT_NAME
Var1ExtRefStat = EXT_REF_STATUS
SET_LIST = Set1;
```

The Value Variable is assigned to the Data Access Set Set1. Because there is a Point Name Alias Variable defined (Var1Point_Name), the Alias Variable is included in the Dynamic Selection Set that is also named Set1.

The 'ALL' selection set

Both Data Access Sets and Dynamic Selection Sets have an implicit ALL set. The ALL Data Access Set includes all Value Variables in the application. The ALL Dynamic Selection Set includes all Alias Variables in the application. An EXEC DDA operation can operate on the ALL selection set by specifying 'ALL' enclosed in single quotation marks. This set name is reserved by OpenDDA and cannot be used to name any other sets.

9.3 How Dynamic Selection of Data Works

The basic concept

The following are the basic steps for dynamic selection of data:

- You identify the External Reference components that will be dynamically selected at run time (these components can be point names, parameter names, index values, or any combination thereof)
- You organize these components into one or more sets that will be acted upon as a group
- You assign an Alias Variable name for each of the External Reference components that will be dynamically selected, and use these alias names in the External Data Statements in the EDB
- You build your application in the normal way
- At run time, your program dynamically selects data by placing desired External Reference names in Alias Variables and then executing an `EXEC DDA MODIFY CIO`
- The next `EXEC DDA READ` or `EXEC DDA WRITE` reads and/or writes from/ to the new external locations

The CIO file is not changed

The `EXEC DDA MODIFY CIO` command modifies a memory resident copy of the CIO file (the currently active CIO). It does not modify the disk resident copy of the CIO file. (The CIO file can be modified with the `EXEC DDA WRITE CIO` command—see Section 8 “Remapping CIO Files”)

9.4 Identifying Dynamic External Reference Components

Specifying a static External Reference

The External Reference for a Value Variable is specified in the Location Specification of an External Data Statement. For example:

```
Var1 = 1.0 /REAL  
Var1Status = INPUT CURRENT (AXM, A100.PV)
```

The External Reference, A100.PV, is explicitly stated in this example, and cannot be dynamically selected at run time.

Specifying a dynamic External Reference

Any of the components of an External Reference (point name, parameter name, index name) can be designated for dynamic selection. This is done by using an asterisk (*) character in the External Reference specification. For example:

```
Var1Status = INPUT CURRENT (AXM, *.PV)
```

In the above example, the point name component of the External Reference is designated for dynamic selection.

```
Var1Status = INPUT CURRENT (AXM, A100.*)
```

In the example above, the parameter name component will be dynamically selected.

```
Var1Status = INPUT CURRENT (AXM, *.* )
```

In the example above, both point name and parameter name will be dynamically selected.

```
Var1Status = INPUT CURRENT (AXM, A100.COEFF(*) )
```

In the example above, the index name component will be dynamically selected.

When the application is built

When you build your application, what happens to External Reference components that are defined with "*" instead of actual names? These components are initialized in the CIO to DDA_NULL. This tags those External Reference as incomplete, and subsequent read or write operations will not attempt to access data for those references.

Before data can be accessed, the NULL references must be replaced with valid names. This is done in the application by the process of dynamic selection—loading the Alias Variables with desired names and executing the EXEC DDA MODIFY CIO statement.

9.5 Alias Variable

Overview

The External Reference components that were defined with "*" must be assigned valid external names before your application can access external data for those Value Variables. This is done through the use of Alias Variables included in External Data Statements in the EDB, and through the EXEC DDA MODIFY CIO statement.

Alias Variables are variables associated with the External Reference components defined by "*". The application loads these variables with valid External Reference names, and then executes the EXEC DDA MODIFY CIO statement which generates the appropriate external addresses in the memory resident CIO file.

syntax

The syntax is as follows:

```
alias_variable = external_ref_keyword
```

where external_ref_keyword is one of the following:

- POINT_NAME
- PARAM_NAME
- INDEX_NAME
- FULL_NAME

Alias Variable examples

The following examples illustrate the use of Alias Variables with External Reference components that are defined by "*" characters.

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.PV)
Var1Point_Name = POINT_NAME
SET_LIST = Boiler1;
```

In the above example, the point name component of the External Reference is designated for dynamic selection by "*". Therefore, an Alias Variable is defined using the POINT_NAME keyword. The Alias Variable is included in the Dynamic Selection Set "Boiler1".

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.* )
Var1Point_Name = POINT_NAME
Var1Param_Name = PARAM_NAME
SET_LIST = Boiler1;
```

In the example above, both point name and parameter name will be dynamically selected. Therefore, two Alias Variables are defined using the POINT_NAME and the PARAM_NAME keywords.

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.*(*) )
Var1Point_Name = POINT_NAME
Var1Param_Name = PARAM_NAME
Var1Index_Name = INDEX_NAME
SET_LIST = Boiler1;
```

In the example above, both point name, parameter name, and index name will be dynamically selected. Therefore, three Alias Variables are defined using the POINT_NAME, PARAM_NAME, and INDEX_NAME keywords. Compare this syntax with that of the FULL_NAME variable, which follows.

Using the FULL_NAME keyword

An External Reference that is specified using "*" characters for all components (point, parameter, and index) can be associated with a single Alias Variable defined with the FULL_NAME keyword.

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.*(*) )
Var1Point_Name = FULL_NAME
SET_LIST = Boiler1;
```

In the above example, the point name, parameter name, and index components of the External Reference are designated for dynamic selection by "*". The External Reference consists of more than one component that will be dynamically selected. However, it is considered to be a single External Reference.

A full name Alias Variable must be specified with three asterisks: *.*(*). However, when the Alias Variable is loaded with an External Reference name, it can be of either the point.parameter or point.parameter(index) form. For example, Var1Point_Name in the above example could be loaded with either:

```
SIM1.PV, or
pt1.param1(4)
```

The procedures for loading Alias Variables in either FORTRAN or C will be covered in the next subsection.

9.6 Loading Alias Variables

Introduction

The material in this subsection is a duplication of material presented in the *OpenDDA Reference Manual*. It is included here in its entirety because all of the concepts presented are necessary for an understanding of the Dynamic Selection Sets functionality—even at an overview level.

Overview

After an OpenDDA application is initially loaded into memory and the `EXEC DDA INITIALIZE` statement is executed, the OpenDDA memory resident CIO contains the `DDA_NULL` value for all External Reference components associated with Alias Variables (unless an `EXEC DDA MODIFY CIO` and `EXEC DDA WRITE CIO` had been executed and the CIO saved in a previous execution of the application). Therefore, no external data can be accessed for these Value Variables. To select External References, the OpenDDA application needs to:

- load valid External Reference components into the Alias Variables, and
- execute an `EXEC DDA MODIFY CIO` statement.

This subsection describes how to load External Reference components into the Alias Variables.

Saving the CIO

Once External Reference components are loaded into the OpenDDA memory resident CIO, they can be saved to disk for future usage by using the `EXEC DDA WRITE CIO` function (see Section 5 of the *OpenDDA Reference Manual*).

Persistence of External References

This procedure of modifying the External Reference Components, saving the memory resident CIO to disk, reloading the CIO later, and using the External References is known as "persistence of External References." The values for all External References at the time of saving the memory resident CIO to disk will be restored when the CIO is read into memory using the `EXEC DDA READ CIO` statement. In this manner, the dynamically selectable External References can be preserved for a later time by writing them to the CIO on disk.

External Reference components are strings

Point Name, Parameter Name, Index Name, and Full Name Alias Variables are ASCII strings. To load an External Reference component into the Alias Variable, simply use the constructs provided by the high level language to place an ASCII string into the Alias Variable. The ASCII string that you place into the Alias Variable should be identical to the ASCII string that you would use in an External Data Statement to indicate an External Reference.

C examples

All Alias Variable strings are null-terminated. Null strings will be handled as invalid references and are subject to the platform-specific resolution error handling.

```
strcpy(Point_Alias_Variable, "A100");  
strcpy(Parameter_Alias_Variable, "COEFF");  
strcpy(Index_Alias_Variable, "2");  
strcpy(Full_Name_Alias_Variable, "A100.COEFF(2)");
```

FORTRAN examples

All Alias Variable strings should adhere to the normal FORTRAN rules for strings. Strings that are smaller in size than the declared string variable must be blank-filled.

```
Point_Alias_Variable = 'A100'  
Parameter_Alias_Variable = 'COEFF'  
Index_Alias_Variable = '2'  
Full_Name_Alias_Variable = 'A100.COEFF(2)'
```

Loading DDA_NULL

If an Alias Variable is loaded with the predefined value DDA_NULL, an EXEC DDA MODIFY CIO does not treat this as an error condition. It will **not** force the overall status from the MODIFY to be HGSC_CHECK_STATUS. The External Reference Status Variable will contain DDA_EXTREF_INCOMPLETE. If a read or write is attempted, the value/store status will contain DDA_EXTREF_NULL. A NULL reference is treated as one that is not being used at this time. Therefore, the overall status of the call is not flagged with an error, but the individual status (external reference, value, store) variables return sufficient information for the user to detect NULL references at run time.

Dynamic components of External References are initialized to DDA_NULL when the application is built. This provides the programmer with the flexibility to define references in the EDB for future expansion of the application. The EXEC DDA LOAD ALIAS statement, which will be described later, can be used to bring in external component names from the CIO into the high level language Alias Variables, and then those loaded with DDA_NULL will not cause errors when EXEC DDA MODIFY CIO is executed.

9.7 Optional EDB Variables

Overview

There are two optional EDB variables that can be used to support Dynamic Selection Sets functionality. They are:

- External Reference Status Variable
- Modify Code Variable

External Reference Status Variable

The External Reference Status Variable does for dynamic selection what the Value Status and Store Status Variables do for data access. In data access, if a read or write operation returns an overall status of HGSC_CHECK_STATUS, the individual Value Status and Store Status Variables can be checked to determine which reference caused the problem. In dynamic selection, if a CIO modify operation returns an overall status of HGSC_CHECK_STATUS, the individual External Reference Status Variables can be checked to determine which reference caused the problem.

The syntax of the External Reference Status Variable is:

```
ext_ref_status_variable = EXT_REF_STATUS
```

This status variable can return the following values:

- HGSC_VALID—This reference was successfully modified
- DDA_EXTREF_INVALID—The External Reference name was invalid (for example, contained invalid characters)
- DDA_EXTREF_INCOMPLETE—This reference had one or more Alias Variables that were not loaded (contained DDA_NULL)
- DDA_EXTREF_INDETERMINATE—This reference was indeterminate due to an error (for example, an invalid Modify Code)
- Other platform specific status

Example:

```
Var1 = 1.0 /REAL  
Var1Status = INPUT CURRENT (AXM, *.*(*) )  
Var1Point_Name = POINT_NAME  
Var1Param_Name = PARAM_NAME  
Var1Index_Name = INDEX_NAME  
Var1_Ext_Ref_Status = EXT_REF_STATUS  
SET_LIST = Boiler1;
```

The above example is valid. The point name, parameter name, and index name variables combine to comprise a single External Reference. Var1_Ext_Ref_Status is the location that will receive status information resulting from an EXEC DDA MODIFY CIO.

Modify Code Variable

The optional EDB Modify Code Variable gives you individual control over whether or not an External Reference will be modified by an EXEC DDA MODIFY CIO operation. The syntax of the Modify Code Variable is:

```
modify_code_variable = MODIFY_CODE
```

It is an integer variable that can be loaded with either of two predefined Modify Codes:

- **DDA_MODIFY**—This External Reference will be modified by an EXEC DDA MODIFY CIO operation
- **DDA_KEEP**—This External Reference will not be modified by an EXEC DDA MODIFY CIO operation

If a Modify Code variable is not specified, the default DDA_MODIFY will be used.

Example:

```
Var1 = 1.0 /REAL  
Var1Status = INPUT CURRENT (AXM, *.*(*) )  
Var1Full_Name = FULL_NAME  
Var1_Modify_Code = MODIFY_CODE  
SET_LIST = Boiler1;
```

Following are some examples of valid code to load the Modify Code Variable.

```
Var1_Modify_Code = DDA_MODIFY
```

In this example, the Modify Code Variable is being set to the Modify Code that will enable modification of the External Reference by an EXEC DDA MODIFY CIO.

```
Var1_Modify_Code = DDA_KEEP
```

In this example, the Modify Code Variable is being set to the Modify Code that will "keep" the External Reference unchanged by an EXEC DDA MODIFY CIO.

9.8 Arrays and Indexes

Overview

OpenDDA provides support for Value Variable arrays and Alias Variable arrays. This functionality, when used with FULL_NAME and INDEX_NAME Alias Variables, allows the application to access multiple External References, or multiple values from an external array, with a single READ or WRITE statement. We will illustrate some of this capability with examples; however, for a fuller understanding, the reader should refer to the *OpenDDA Reference Manual*. Most of the applicable reference material is contained in the Section 5, “Dynamic Selection Variables”, and, more specifically, in subsection 5.5 which deals with Alias Variable Arrays, and subsection 5.7 which covers Indexes and Dynamic Selection.

Accessing multiple references

By using a Value Variable array and an Alias Variable array, the programmer can access multiple External References and read or write the values to or from the Value Variable array.

- The External References desired are loaded into the elements of the Alias Variable array.
- Each External Reference must be a scalar—a point.parameter (such as SIM1.PV), or a point.parameter(index) specifying an element of an external array (such as my_array.param1(5)).
- The following rules apply:
 - The Alias Variable must be declared as FULL_NAME in the EDB.
 - The Value Variable array and the Alias Variable array must be of the same size (although the subrange specifications do not have to agree).

Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*) )
Var1Full_Name[0:9] = FULL_NAME
SET_LIST = Boiler1;
```

In this example, the Alias Variable array Var1Full_Name[0:9] is loaded with 10 independent External References. After an EXEC DDA MODIFY CIO operation, an EXEC DDA READ will access the 10 External Reference values and place them in the Value Variable array Var1[0:9]. The index is optional in each element of the Alias Variable array that specifies the actual External Reference. If present, it is used to access a single element of an external array. If not present, a scalar is referenced.

Accessing multiple values from an external array

OpenDDA allows you to access multiple consecutive elements of an external array using a Value Variable array to receive READ data or hold WRITE data. A scalar Alias Variable is used to reference, or "anchor" the starting location of the external array from which consecutive elements of data will be read or written. The Alias Variable must be declared as either FULL_NAME or INDEX_NAME. An index is required to identify which element of the external array is the starting element (anchor).

FULL_NAME Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*) )
Var1Full_Name = FULL_NAME
SET_LIST = Boiler1;
```

In the above example, the Value Variable Var1[0:9] is an array. The FULL_NAME Alias Variable Var1Full_Name must contain an index in order to indicate the anchor point in the external array. An EXEC DDA READ will cause Var1[0:9] to be given 10 sequential values starting (anchored) at the location specified by the index in the Var1Full_Name Alias Variable.

INDEX_NAME Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM,pt1.parm1(*) )
Var1Index_Name = INDEX_NAME
SET_LIST = Boiler1;
```

The above example is similar to the previous example except that an INDEX_NAME Alias Variable is used to determine the starting point (anchor) to READ from the pt1.parm1 array.

9.9 MODIFY CIO Statement

Overview

The EXEC DDA MODIFY CIO statement selects new External References for items that are members of the specified Selection Set or Sets. The Alias Variables associated with the Selection Set(s) must be loaded with the desired External Reference names prior to invocation of this statement. The Modify Code Variable associated with each External Reference must be loaded with or must already contain the desired value.

Syntax

```
EXEC DDA MODIFY CIO,  
    SET = (dss_1 [, dss_2, ...] ),  
    STATUS = exec_status;
```

dss_1, dss_2,... are Dynamic Selection Sets.

If Modify errors are encountered

For references that are indeterminate or invalid, or for other External Reference validation errors, the overall status returned is HGSC_CHECK_STATUS, and:

- The External Reference Status Variable should be checked for additional information.
- The External Reference associated with the Alias Variable that caused the error will be modified (if possible—for example, if the Modify Code is bad, the reference will not be changed).
- An EXEC DDA READ or EXEC DDA WRITE will return DDA_EXTREF_NOTBOUND for the value_status or store_status denoting that there is no External Reference binding for the associated value_variable.
- An EXEC DDA READ or EXEC DDA WRITE will not access external data for any Value Variable associated with the Alias Variable that caused the error.

If Alias Variables are NULL

For references that are not complete (have NULL components):

- The External Reference Status does not have to be checked for additional information, but the user can check it to determine which references are NULL.
- The External Reference associated with the Alias Variable that is NULL will be modified with NULL components.
- An EXEC DDA READ or EXEC DDA WRITE will not access external data for any Value Variable associated with the Alias Variable that is NULL. The associated Value Variable is unchanged.

- If an Alias Variable contains the value DDA_NULL, an EXEC DDA MODIFY CIO will return DDA_EXTREF_INCOMPLETE in the External Reference Status Variable. This is not an error condition, however, and will not impact the overall status of the modify.
- An EXEC DDA READ or EXEC DDA WRITE will return DDA_EXTREF_NULL for the value_status/store_status denoting that there is no External Reference binding for the associated Value Variable. This is not an error condition, however, and will not impact the overall status of the read or write (will not force HGSC_CHECK_STATUS).

It is important to understand that NULL references are not considered errors. If you wish to set a reference unusable and have errors reported during both the EXEC DDA MODIFY CIO and EXEC DDA READ or EXEC DDA WRITE operations, you could use a "blank" string. This would be flagged as an error (DDA_EXTREF_INVALID) during all subsequent actions as compared with a Null reference which is not considered to be an error (DDA_EXTREF_INCOMPLETE).

Example

The following is an example of an EXEC DDA MODIFY CIO statement:

```
EXEC DDA MODIFY CIO,  
    SET = 'Boiler1',  
    STATUS = exec_status;
```

All Alias Variables in the Dynamic Selection Set 'Boiler1' will be modified unless they have Modify Code variables set to DDA_KEEP. If the exec_status returns HGSC_VALID, all references were successfully modified. If the status is not HGSC_VALID, the individual External Reference Status Variables can be checked to determine which reference(s) caused the problem.

9.10 Optional EXEC DDA Statements

Overview

There are two additional EXEC DDA statements that are provided to support dynamic selection functionality. They are:

- EXEC DDA LOAD ALIAS
- EXEC DDA LOAD MODIFY_CODE

LOAD ALIAS statement

The EXEC DDA LOAD ALIAS statement loads all Alias Variables and associated External Reference Status Variables for the specified Dynamic Selection Set or Sets with the values that are stored in the currently active memory resident CIO.

Syntax:

```
EXEC DDA LOAD ALIAS,  
    SET = (dss_1 [,dss_2...] ),  
    STATUS = exec_status;
```

where dss_1, dss_2, ... are the names of Dynamic Selection Sets enclosed in single quotes.

Some of the potential uses of this statement are:

- The application can query, or verify, the current External References.
- If variable names are overloaded (for example, same names used in multiple External Data Statements that belong to different sets), the application can restore all of the values for a specific set with one call.
- If the application uses multiple CIOs, it can select a new memory resident CIO and then block load all variables for that CIO with one call.
- It can provide a mechanism to support incremental changes if the user does not desire to use Modify Codes or Selection Sets. This is achieved by back loading (with LOAD ALIAS) Alias Variables, changing components desired, and then forward loading all references (with MODIFY CIO).

Example:

```
EXEC DDA LOAD ALIAS,  
    SET=('SS1', 'SS2'),  
    STATUS=exec_status;
```

After execution of this statement, all Alias Variables and the associated External Reference Status Variables for the Dynamic Selection Sets 'SS1' and 'SS2' will be loaded with the values contained in the currently active memory resident CIO (values established by the most recent modify operation for those sets, or by the INITIALIZE if no modify statements have been executed). See the discussion of the EXEC DDA LOAD ALIAS statement in subsection 6.10 in the *OpenDDA Reference Manual* for a more complex example illustrating overloading.

LOAD MODIFY_CODE statement

This statement will load all defined Modify Code variables with a user- specified Modify Code. This execution statement is provided to allow applications to easily "batch" load all Modify Code variables with a specific Modify Code.

Syntax:

```
EXEC DDA LOAD MODIFY_CODE,  
      VALUE = modify_code_value,  
      STATUS = exec_status;
```

Note: `modify_code_value` can be either of the following:

- One of the Modify Code Value constants `DDA_MODIFY` or `DDA_KEEP`.
- A native high language variable that is loaded with a Modify Code constant prior to executing the `EXEC DDA LOAD MODIFY_CODE` statement.

This statement acts on the set 'ALL'. All Modify Code variables in the application will be loaded. If the Modify Code Variable is an array, all elements of the array will be loaded.

Usage

The `EXEC DDA LOAD MODIFY_CODE` statement can be used to effect incremental changes of External References. For example, the `EXEC DDA LOAD MODIFY_CODE` could be used to load all Modify Code variables with `DDA_KEEP`. Then certain variables could be loaded with `DDA_MODIFY`. A subsequent `MODIFY CIO` would then modify only those references whose Modify Code was set to `DDA_MODIFY`.

Example:

```
EXEC DDA LOAD MODIFY_CODE,  
      VALUE = DDA_KEEP,  
      STATUS = exec_status;
```

After execution of the above statement, all Modify Code variables in the application will be set to the value `DDA_KEEP`.

10. Cache Maintenance Tool

10.1 Overview

Background

When you build an OpenDDA application, one of the steps is to parse the EDB. As part of this process, OpenDDA accesses the control platform (for example, LCN) and performs the following.

- It validates that the External References specified in the EDB exist.
- It resolves the External References by determining the internal ID that will be used to access the External References in future EXEC DDA READ and EXEC DDA WRITE operations (the internal ID is used for performance reasons).
- It builds the Control Input Output (CIO) file that the application will use to convert from the ASCII representation of the External References to the internal ID form when EXEC DDA READ and EXEC DDA WRITE operations are performed.

Purpose of the cache

The process of validating and resolving External References by accessing the control platform is relatively slow. Building a large application (one with many External References) can take a considerable amount of time. For applications with static external data, this delay is a one-time occurrence. However, if Dynamic Selection is used, External References are changed at run time by modifying the CIO (see Section 9), and the new references must be resolved. Therefore, as OpenDDA resolves External References, the results are stored in cache entries. When a new application is built, or when data is Dynamically Selected at run time, OpenDDA first checks the cache to resolve External References before going out to the platform. Therefore, the cache enhances performance because External References only need to be resolved once. This is beneficial in these cases:

- when creating or modifying an application that uses existing External References, and
- when using Dynamic Selection of External References at run time, and
- at run time when writing External IDs.

Purpose of the Cache Maintenance Tool

The Cache Maintenance Tool (CMT) is provided to allow the user to examine and/or delete entries in the cache and to back up the cache.

10.2 Characteristics of the Cache

The cache is memory mapped

The cache file is a disk resident file that stores information related to External References. At run time, the disk resident file is mapped into virtual memory, thereby providing a persistent image of the cache file. Because it is a memory mapped file, the memory needed to contain an image of the cache file is allocated when an OpenDDA application (including CMT) executes. This memory mapped file is shared by all OpenDDA applications that are running at that time. When the last OpenDDA application terminates, this memory is freed.

Cache files

The DDACache file is created under the `/opt/DDA` directory during the install procedures of OpenDDA. The executable is in `/opt/DDA/bin`.

Cache size

The DDACache file is created with a default size of 50,000 entries (approximately 5 Mb). If the cache file starts to get filled up, the cache file will grow in increments of 10,000 entries (approximately 1 Mb) at a time. This will continue until either there isn't enough memory available or the cache file size has reached 1,000,000 entries (approximately 100 Mb). If the cache file reaches the limit and cannot grow further, errors will not be returned when accessing the cache for points not in cache. Instead, all calls to retrieve point type information from points not in the cache will use an LCN access to fetch this information every time that point is accessed. This will cause an increase in the time that it takes for the OpenDDA application to execute. The administrator can use the CMT tool to delete entries in the cache to allow new cache entries again.

If the cache is deleted

If the DDACache file is deleted for any reason, it will be created the next time an OpenDDA application is executed or when the CMT tool is invoked. However, this new file will be created with permissions only to the user id that built it. The permissions on the DDACache file must be changed to allow other users to read and update this file. Use the following command to change the permissions:

```
chmod 666 /opt/DDA/DDACache
```

Error creating cache file

If there is an error creating a new DDACache file or accessing the current file using the Cache Maintenance Tool, the following error message will appear:

```
cmt: error unable to access cache
```

If this happens, check to see if the DDACache file exists and check the access rights on that file. If you are trying to create a new DDACache file and you receive this error, check to see if you have the proper "owner" or "group" privileges to be able to create the file in `/opt/DDA/`.

Cache entries

Cache entries contain both the external and internal representations of LCN point and parameter names and other information as follows:

- **Point ID**—this field is broken up into several subfields containing various pieces of information about the entity on the LCN.
- **Parameter ID**—this field contains the internal identifier for the parameter on the External Reference.
- **Data Type**—this field contains the LCN data type of the cache entry
- **Start Index**— if the data type is an array, this field contains the starting index of the array
- **Array Size**—if the data type is an array, this field contains the number of elements in the array
- **String Length**—if the data type is an ASCII string, this field contains the length of the string
- **Enumeration ID**—if the data type is an enumeration, this field contains the ID of the enumeration (ordinal value)
- **Set ID**—if the data type is an enumeration, this field contains the ID of the enumeration set (an internal identifier)
- **Entry Status**—this field contains "valid" and is reserved for future use
- **Entry Time Stamp**—this field contains the time that the cache entry was last accessed

Cache entry freshness

Cache entry freshness is determined by comparing the time stamp field in the cache entry with the current time. This difference is then compared with a delta (freshness) time specified by the user for commands that involve freshness. Examples will illustrate this concept.

10.3 Invoking the Cache Maintenance Tool

Invoking CMT from the shell prompt

From the shell prompt, enter any valid CMT command:

```
$ cmt show entry "*"
```

Redirection and piping

You can redirect or pipe commands to CMT. When executing CMT commands in this manner, you do **not** put the External Reference Name in double quotation marks.

Examples:

For the purpose of these examples, the file `commands.cmt` contains the following:

```
show entry /brief ss\dbmac_n.pv
show entry /brief ss\perf.numa1
```

Redirection example:

Command line:

```
cmt < commands.cmt
```

Output:

```
SS\DBMAC_N.PV
1 Entries Found
SS\PERF.NUMA1
1 Entries Found
```

Piping example:

Command line:

```
cat commands.cmt | cmt
```

Output:

```
SS\DBMAC_N.PV
1 Entries Found
SS\PERF.NUMA1
1 Entries Found
```

10.4 Cache Maintenance Tool Commands

Command list

The following is a list of CMT basic commands:

- **show**—this command is used to display cache information to standard output
- **delete**—this command is used to remove cache entries
- **checkpoint**—this command is used to create a snapshot copy of the cache file

Options

The following options can be used with either the **show** or **delete** commands:

- **entry**—this option is used to either display or delete one or more cache entries that correspond to a user supplied External Reference name or names
- **freshness**—this option is used to either display or delete cache entries that do not meet a user specified freshness criteria

The following option can only be used with the **show** command.

- **cache**—used to display cache statistics

Each **show** and **delete** command must include one (and only one) option.

Summary of commands

The following is a summary of CMT commands:

```
show entry external_reference_name [/full | /brief]
show freshness freshness_time [/full | /brief]
show cache
delete entry external_reference_name
delete freshness freshness_time
checkpoint pathname
```

Where:

- *external_reference_name* is an ASCII [ng\]point.parameter name
- *freshness_time* is an ASCII delta time value
- *pathname* is the pathname where the cache snapshot will be stored

The vertical bar "|" denotes that either of the qualifiers, but not both, can be specified.

The brackets [] denote an optional item.

Qualifiers

The following optional qualifiers can be used with the **show** command:

- **/full**—this qualifier will display the entire contents of cache entries
- **/brief**—this qualifier will display only the ASCII External Reference

The default qualifier values depend on:

- the show command option specified on the command line, and
- whether or not the External Reference is specified using a wildcard (*) character or characters.

Default qualifier values are shown in the following table.

Table 10-1 Default Qualifier Values

Show Command Option			
Entry		Freshness	Cache
Wildcard(s)	No Wildcards		
/brief	/full	/brief	N/A

You can override any default value by explicitly specifying a qualifier on the command line.

Both qualifiers are position independent and can occur in any position on the command line.

external_reference_name

An ASCII External Reference that is specified by the user to search the cache during a `show entry` or `delete entry` command may consist of either a specific `ng\point.parameter` name, or any part or parts of the name may contain a wildcard character (*). If no ASCII External Reference is supplied on the command line, the user is prompted one time for input. ASCII External References are limited to 50 characters.

If no wildcards are used in the ASCII External Reference specified in a `show entry` command and `/brief` is not explicitly specified, all of the information contained in the cache entry is displayed. If no cache entry is found, nothing is displayed.

ATTENTION

The *external_reference_name* argument should be enclosed in double quotation marks when entering a command from the shell prompt:

```
cmt show entry "fw\100.pv"
```

Do not use quotation marks when using redirection from a file or when piping a command file into CMT.

freshness_time

Cache entries with a time stamp that is older than the freshness value specified by the user are either displayed or deleted by CMT commands. The user specified freshness value is entered as an ASCII delta time of the format

ddd:hh:mm

where ddd is the number of days (0-365), hh is the number of hours (0-23), and mm is the number of minutes (0-59). If no ASCII delta time is supplied on the command line, the user is prompted one time for input.

When a **show freshness** command is requested by the user, only the ASCII entry name is displayed unless the **/full** qualifier is included in the command line. If no cache entries are found to be outside the freshness time, nothing is displayed.

Checkpoint/ Restore

The DDACache file can be checkpointed to a secondary file by using the checkpoint command:

```
cmt checkpoint /mydir/cache.bak
```

The maximum allowed length of the file name, including path, is 74 characters. If this limit is exceeded, an error message will be given by CMT.

To restore the backup file to be the DDACache file, use the HP-UX copy command (cp):

```
cp /mydir/cache.bak /opt/DDACache
```

WARNING: Copying over the DDACache file in **/opt/DDA** while other users are accessing it may cause them to access the new file, giving them erroneous results. This is because the copy will overwrite any existing DDACache file regardless of whether or not it is being accessed.

show cache command

This command is used to show statistical information about the cache file. It should be used by the administrator to determine how much space he has available in the cache file. If the cache file is starting to run out of space, this command will indicate how much is available. The administrator can then take corrective action (e.g. delete old entries) to free up more space.

The command displays the following information:

- **Current Number of Elements**—This indicates the number of point.parameter entries that are currently stored in the Cache file.
- **Current Minimum Elements** —This indicates the least number of entries that were ever stored in the Cache file. This should always be 0.
- **Current Maximum Elements**—This indicates the largest number of entries that were ever stored in the Cache file.
- **Cache Extend Elements**—This indicates the number of elements by which the Cache file will be incremented as it fills up and needs more space.
- **Cache Minimum Elements**—This indicates how many entries the Cache file will be able to store when it was created until it has to start increasing the physical file size of the Cache file.
- **Cache Maximum Elements**—This indicates the maximum number of entries that the Cache file will be able to store if it grows to its maximum physical file size.
- **Current Pool Class Revision and Current pdaSMap Class Revision**— These indicate the revision of the software that is used to create and access the Cache file. This information should be supplied to TAC if problems are encountered accessing the Cache file.

Case of commands and qualifiers

CMT commands, options, and qualifiers are not case sensitive. If CMT is unable to parse a command line, it displays a message similar to the following:

```
cmt: error parsing command line <Shwo Entry fe\*.pv>
```

In the example above, the error resulted because "Show" was misspelled.

Command line limits

A CMT command line has a limit of 210 characters, whether it is executed directly from the UNIX shell or via redirection from a file. Each string on the command line, delimited by a blank space, has a limit of 50 characters.

The maximum number of arguments (everything on the command line after “cmt”) varies for each command. The show command has a maximum of 4 arguments (e.g., `cmt show entry <entryname> <qualifier>`). The delete command has a maximum of 3 arguments (e.g., `cmt delete freshness <freshnessvalue>`). Finally, the checkpoint command has a maximum of 2 arguments (e.g., `cmt checkpoint <filename>`).

If any of these limits are exceeded, the program will exit.

Delete confirmation options

If delete commands are processed interactively, the user is prompted for confirmation. Example:

```
Delete FE\A100.PV [y(es), n(o), c(ancel), a(ll)][n]?
```

The prompt includes the ASCII External Reference name along with four possible responses. The four responses are:

- **y**—by responding with a "y" the entry specified in the prompt will be deleted.
- **n**—by responding with an "n" the entry specified in the prompt will not be deleted. This is also the default—simply pressing the RETURN key is equivalent.
- **c**—by responding with a "c" the entry specified in the prompt will not be deleted and the delete command will be terminated.
- **a**—by responding with an "a" the entry specified in the prompt will be deleted and *all* subsequent matching entries will be deleted without further prompts.

If you are using piping or redirection, confirmation is not requested.

10.5 Examples

show entry examples

The following example illustrates how to display a single cache entry. Note that detailed information is given because the default qualifier is `/ full` when a specific External Reference is used.

```
cmt show entry "fw\A100.pv"
```

```
Entry: FW\A100.PV
Point ID:
Entity Type: 1
Unit Association: 10
LCN Number: 4
Function Set: 2
Data Realm: 5
Revision Number: 3
Local Routing: 7
Parameter ID: 47
Data Type      Real
Enumeration ID 0
Set ID         0
Start Index: 0
Array Size: 0
String Length: 0
Entry Status:  valid
Time Stamp    Fri May 26 13:22:14 1995
```

The next example illustrates how to display multiple cache entries using wildcards. Specifically, all of the entries associated with a given LCN and parameter name are requested.

```
cmt show entry "fe\*.sp"
```

```
FE\PT100.SP
FE\STDPT.SP
FE\V1128.SP
. . .
. . .
```

The following example shows the case when an LCN name or wildcard is NOT specified. The CMT will automatically search entries in the cache, which may exist on more than one LCN, that match the entry given on the command line. Therefore, all occurrences of that entry on all LCN's which are present in the cache will be returned in the result of the query. For example, if the query was:

```
cmt show entry a100.pv
```

and the following were some of the entries in the cache:

```
SS\a100.pv
LP\b100.pv
LP\a100.pv
HG\c100.op
```

then the result of the query would be:

```
SS\a100.pv
LP\a100.pv
```

Whether or not the LCN name (NG specifier) is given, if there IS a wildcard character ('*') in the external_reference_name, then the output will be given in /brief format. If there is no wildcard, the output will be given in /full format.

show freshness example

The following example shows how to display cache entries which have been in the cache for over the user specified time. Specifically, all of the entries over 10 hours old are requested.

```
cmt show freshness 0:10:0
```

```
FE\A100.PV
SS\A200.PV
FW\A100.PV
. . .
. . .
```

show cache example

This example shows how to display cache statistics.

```
cmt show cache
Current Number of Elements: 147
Current Minimum Elements:  0
Current Maximum Elements:  147

Cache Extend Elements:  10000
Cache Minimum Elements: 50000
Cache Maximum Elements: 1000000

Current Pool Class Revision: 3
Current pdaSMap Class Revision: 1
```

delete entry examples

The following example illustrates how to delete a single cache entry.

```
cmt delete entry "fe\b100.pv"
Delete FE\A100.PV [y(es), n(o), c(ancel), a(ll)][n]? y
Entry FE\B100.PV deleted
```

The next example illustrates how to delete multiple cache entries using wildcards. Specifically, all of the entries associated with a specific LCN are requested to be deleted.

```
cmt delete entry "ss\*"
Delete SS\A100.PV [y(es), n(o), c(ancel), a(ll)][n]? y
Entry SS\A100.PV deleted
Delete SS\A200.PV [y(es), n(o), c(ancel), a(ll)][n]? n
Delete SS\B100.PV [y(es), n(o), c(ancel), a(ll)][n]?
Delete SS\B200.PV [y(es), n(o), c(ancel), a(ll)][n]? a
Entry SS\B200.PV deleted
Entry SS\B300.PV deleted
. . .
. . .
```

The following example shows the delete entry command being initiated with no NG name on the entry specification:

```
cmt delete entry a100.pv
```

and the following entries exist in the cache:

```
SS\a100.pv
LP\b100.pv
LP\a100.pv
HG\c100.op
```

the CMT would try to delete both SS\a100.pv AND LP\a100.pv. The user would be prompted for confirmation. Refer to “Delete confirmation options” in subsection 10.4.

delete freshness example

The following example shows how to purge the cache of stale entries. Specifically, all of the entries over three days old are requested to be deleted.

```
cmt delete freshness 3:0:0
Delete FE\A100.PV [y(es), n(o), c(ancel), a(ll)][n]? y
Entry FE\A100.PV deleted
Delete FE\A200.PV [y(es), n(o), c(ancel), a(ll)][n]? y
Entry FE\A200.PV deleted
Delete FW\A100.PV [y(es), n(o), c(ancel), a(ll)][n]? y
Entry FW\A100.PV deleted
Delete TH\A100.PV [y(es), n(o), c(ancel), a(ll)][n]? c
```

checkpoint example

The **checkpoint** command is used to make a snapshot copy of the cache file. This copy can be used as a backup. The command is invoked as follows:

```
cmt checkpoint /users/mydir/cache.copy
```

This command will create a snapshot copy of the current cache file into `cache.copy` in the directory `/users/mydir`.

Restore example

To restore the cache using the backup copy created above, you would use the HP-UX copy command:

```
cp /users/mydir/cache.copy /opt/DDA/DDACache
```

Upon restoring the DDACache file from a checkpoint, verify that the access rights allow other users to access the cache. If the access rights need to be updated to allow other users to access the file, use the following command:

```
chmod 666 /opt/DDA/DDACache
```

NOTE: you must be a superuser (have superuser privileges) in order to change the access privileges.

11. OpenDDA Application Examples

11.1 Application Examples Distributed with the Software

Purpose of application examples

The example applications are meant to help you understand some of the features of OpenDDA and how to use them in an application. Several example applications are distributed with the OpenDDA software in both FORTRAN and C.

Location of application examples

All of the application examples are located in the directory `/opt/DDA/examples`. To experiment with an example, we suggest that you copy the example and its DEF file, if it has one, to your working directory.

Table of examples included

Table 11-1 Application Examples Included with OpenDDA

Application Name	Files Required	Description
Alltestc	alltestc.c alltestc.def	Shows examples of EDB statements, declarations of variables in C, and EXEC DDA statements
Alltestf	alltestf.f alltestf.def	Shows examples of EDB statements, declarations of variables in FORTRAN, and EXEC DDA statements
Cexample	cexample.c cexample.def	Simple C example: changing a setpoint
fexample	fexample.f fexample.def	Simple FORTRAN example: changing a setpoint
ddademo	ddademo.c ddademo.def	A C application that shows the use of basic OpenDDA execution statements—INITIALIZE, LOAD STORE_CODE, READ, WRITE, and TERMINATE
ddademo	ddademo.f ddademo.def	A FORTRAN application that shows the use of basic OpenDDA execution statements—INITIALIZE, LOAD STORE_CODE, READ, WRITE, and TERMINATE
lvrf	lvrf.f lvrf.def	A FORTRAN application example that calculates Liquid Vapor Ratio

Table 11-1 Application Examples Included with OpenDDA

Application Name	Files Required	Description
nox_demo	nox_demo.c	An example that uses NOX variables
read_tank_info_c	read_tank_info_c.c	A C example that uses advanced features of OpenDDA—Event Hibernation, Remap CIO files, and Dynamic Selection Sets
read_tank_info_f	read_tank_info_c.f	A FORTRAN example that uses advanced features of OpenDDA—Event Hibernation, Remap CIO files, and Dynamic Selection Sets
readit	readit.f readit.def	A FORTRAN example that uses Dynamic Selection Sets
remap	remap.c remap2.edb remap3.edb	A C example that uses the Remap CIO functionality

11.2 How to Build the ddademo Application

Introduction

This subsection discusses building and executing the `ddademo` application example.

The `ddademo` application's main purpose is to show a simple example of how to read and write data.

FORTRAN and C programs provided

Both FORTRAN and C versions of the `ddademo` program are provided in the `/opt/DDA/examples` directory. This section uses the FORTRAN version for illustration.

What the ddademo application does

The `ddademo` application performs the following steps:

1. Initializes the OpenDDA application environment, which includes setting up some internal structures and reading the following files into memory:
 - `ddademo.def`
 - `ddademo.cio`
2. Loads the store code for the three points because the DEF file contains
 `INITIAL_VALUES`
 `STORE_CODE=STORE;`
 All three points will be set to `DDASC_STORE`.
3. Writes a text message indicating the demo is beginning.
4. Reads two real data values from the LCN.
5. Changes the real values and writes them back out to the LCN.
6. Writes a text message indicating the demo is complete.
7. Terminates the OpenDDA application environment, which includes freeing up the memory the application environment was using.

Any errors that occur will cause the application to stop and display information about the error.

11.3 Prepare ddademo

Copy files

To get started, copy the `ddademo` source file and definition file to your application directory using the following commands:

```
cp /opt/DDA/examples/ddademo.f /users/ddauser
```

```
cp /opt/DDA/examples/ddademo.def /users/ddauser
```

Replace `/users/ddauser` with your application directory.

Edit EDB section

Refer to 4.5

Next, you will need to edit the EDB section in `ddademo.f` so it will reference points on your LCN. Change the point information indicated below in the EDB section from `ddademo.f`.

```
C  EXTERNAL DATA BEGIN
C
C    EXTDEF (AXM, DEFAULT)
C
C    REAL1_VALUE = 100.0 /REAL
C    REAL1_VALUE_STAT.VS = INPUT CURRENT(, gdx.gdx1(200))    <---
change
C    REAL1_VALUE_STAT.SC = OUTPUT
C    REAL1_VALUE_STAT.SS = STORE_STATUS
C    SET_LIST = REAL_SET;
C
C    REAL2_VALUE = 100.0 /REAL
C    REAL2_VALUE_STAT.VS = INPUT CURRENT(, gdx.gdx1(201))    <---
change
C    REAL2_VALUE_STAT.SC = OUTPUT
C    REAL2_VALUE_STAT.SS = STORE_STATUS
C    SET_LIST = REAL_SET;
C
C    STRING_VALUE = 'THE STRING IS BLANK' /STRING
C    STRING_VALUE_STAT.VS = INPUT CURRENT(, gdx3.strval(200)) <---
change
C    STRING_VALUE_STAT.SC = OUTPUT
C    STRING_VALUE_STAT.SS = STORE_STATUS
C    SET_LIST = STRING_SET;
C
C  EXTERNAL DATA END
```

- Change gdx.gdx1(200) to a REAL read/write point on your LCN.
- Change gdx.gdx1(201) to a REAL read/write point on your LCN.
- Change gdx3.strval(200) to a STRING read/write point on your LCN.

Save your changes.

11.4 Build ddademo

How to build

Refer to 4.3 and 5.11

Next, build the ddademo application using the following command:

```
/opt/DDA/bin/dda -cb ddademo.f
```

You should get output similar to

dda -cb -o ddademo.f

Begin processing EDB...

DDA-S, EDB processing completed successfully.

Begin validating external data...

Processing Statistics for:
ddademo.f

Total EDB Data Definitions Parsed:	3
Total Dynamic Access Records Built:	0
Total Static Access Records Built:	3

Total Records Processed by Platform(s): 3

Total Errors Detected:	0
Maximum Errors Allowed:	20

Total Items designated for Input:	3
Total Items designated for Output:	3

DDA-S, External data validated successfully.

Expanding execution statements...

Read 210 lines, found 0 errors and 0 warnings.
Wrote 477 output lines.

DDA-S, Execution statements expanded successfully.

Files created by build

Refer to Table 6-1

The following files are created by the build

- ddademo.set
- ddademo.set_idx
- ddademo.cio
- ddademo_src.f

Possible errors

If you get a message like this

Begin validating external data...

File: ddademo.f

Line 10 to Line 14:

```
C      REAL1_value = 100.0 /REAL
C      REAL1_value_stat.vs = INPUT CURRENT(,A100.PV)
C      REAL1_value_stat.sc = OUTPUT
C      REAL1_value_stat.ss = STORE_STATUS
C      SET_LIST = real_set;
```

HGSC-E, Entity not found

it indicates the point was not found.

This error message

HGSC-E, Bad entity-parameter name

indicates the point was found, but not the parameter.

11.5 Validate DEF File

How to validate

Refer to 5.12

After you have built the ddademo application, validate the Definition file to be sure it does not contain any syntax errors. Enter the following command:

```
/opt/DDA/bin/dda -dv ddademo.def
```

You should see the following message

```
DDA-S, Application definition file successfully validated.
```

11.6 Compile/Link ddademo

Compile and link

Refer to Table 6-1

Now compile and link your ddademo application using the following command:
`/opt/DDA/examples/dda_create_fortran_executable ddademo`

You should see output similar to the following:

```
ddademo_src.f:  
      MAIN ddademo:
```

The compile and link creates the following files:

- `ddademo_src.o`
- `ddademo`

11.7 Run ddademo Offline

Try to run offline

Refer to 4.8

You may now try to run your application offline using the following command:

ddademo

The ddademo output will be written to the file `ddademo.dda_out`.

If security is set to read/write

Refer to 3.8

If the security parameter is set to read/write, your output should be similar to the following:

LOAD STORE_CODEs was successful

First WRITE of set "REAL_SET" was successful

real1_value = 1.38

real2_value = 2.38

New values being written

real1_value = 12.48

real2_value = 24.58

WRITE of set "REAL_SET" was successful

Second WRITE of set "STRING_SET" was successful

Note: The above output assumes that all of the print flags in the DEF file are set to OFF. If one or more are set to ON you will get additional output.

If security is NOT set to read/write

Refer to 3.8

If the security parameter is NOT set to read/write, your output should be similar to the following:

LOAD STORE_CODES was successful

First STRING_VALUE bad, status= 393322

Check the status code for its error message by entering the following command:

`/opt/DDA/bin/dda_xstatus 393322`

The status code message for 393322 is

SYS-E, permission denied

Note: The above output assumes that all of the print flags in the DEF file are set to OFF. If one or more are set to ON, you will get additional output.

11.8 Install ddademo

Install your ddademo application

Refer to 5.13

To install your ddademo application, enter the following:

```
/opt/DDA/bin/dda -is ddademo
```

You should see the following output:

```
DDA-S, Application installed successfully.
```

Installation will create a symbolic link in the `/users/axm` directory to the following file:

- `ddademo`

You can verify this by entering the following command:

```
ll /users/axm/ddademo
```

where

`ll` = two lowercase L's

You should get output similar to the following:

```
lrwxrwxrwx  1 ddauser amx      32 Jul 12 10:06 /users/axm/ddademo ->
/users/ddauser/ddademo
```

With this link, a CL program can now activate this application. The output will be found in the Application Directory. This is the directory where the application's Executable Image resides. This directory also contains the application's supporting files—the CIO file and the optional DEF file. This directory can also contain the application source file, but this is not required. The Application Directory, by definition, is the directory from which the application is installed.

11.9 Sample File Listings

ddademo listing

The following is a listing of the ddademo application.

```

PROGRAM DDADEMO
IMPLICIT NONE
INCLUDE 'dda_fortran.f'
C
C
C   EXTERNAL DATA BEGIN
C
C       EXTDEF (AXM, DEFAULT)
C
C       REAL1_VALUE = 100.0 /REAL
C       REAL1_VALUE_STAT.VS = INPUT  CURRENT(, gdx.gdxrl(200))
C       REAL1_VALUE_STAT.SC = OUTPUT
C       REAL1_VALUE_STAT.SS = STORE_STATUS
C       SET_LIST = REAL_SET;
C
C       REAL2_VALUE = 100.0 /REAL
C       REAL2_VALUE_STAT.VS = INPUT  CURRENT(, gdx.gdxrl(201))
C       REAL2_VALUE_STAT.SC = OUTPUT
C       REAL2_VALUE_STAT.SS = STORE_STATUS
C       SET_LIST = REAL_SET;
C
C       STRING_VALUE = 'THE STRING IS BLANK' /STRING
C       STRING_VALUE_STAT.VS = INPUT  CURRENT(, gdx3.strval(200))
C       STRING_VALUE_STAT.SC = OUTPUT
C       STRING_VALUE_STAT.SS = STORE_STATUS
C       SET_LIST = STRING_SET;
C
C   EXTERNAL DATA END
C
C   STRUCTURE /STATUS_TYPE/
C       INTEGER*4  VS          ! Value Status
C       INTEGER*4  SC          ! Store Code
C       INTEGER*4  SS          ! Store Status
C   END STRUCTURE
C
C   REAL*4          REAL1_VALUE, REAL2_VALUE
C   RECORD /STATUS_TYPE/ REAL1_VALUE_STAT, REAL2_VALUE_STAT
C
C   CHARACTER*(DDA_STRING_SIZE) STRING_VALUE
C   RECORD /STATUS_TYPE/ STRING_VALUE_STAT
C
C   INTEGER*4 IOSTATUS

```

```
C      Initializing Application Environment
EXEC DDA INITIALIZE STATUS = IOSTATUS;
IF (IOSTATUS .NE. HGSC_VALID) THEN
  PRINT *, 'ERROR DURING INITIALIZE, STATUS=', IOSTATUS
  GOTO 999
ENDIF

C      Load Store Codes
EXEC DDA LOAD STORE_CODE ,SET = 'ALL', STATUS = IOSTATUS;
IF (IOSTATUS.NE.HGSC_VALID) THEN
  PRINT*, 'Return from LOAD STORE_CODE =', IOSTATUS
  GOTO 99
ELSE
  PRINT*, 'LOAD STORE_CODEs was successful'
  PRINT*, ' '
ENDIF

      STRING_VALUE = 'OpenDDA Demo Beginning'

C      Writing STRING_VALUE to LCN
EXEC DDA WRITE SET = 'STRING_SET', STATUS = IOSTATUS;
IF (IOSTATUS .EQ. HGSC_CHECK_STATUS) THEN
C      Since only one point exists in this set, we know the
variable
C      that had the error.
C
  PRINT *, 'First write of STRING_VALUE bad, status=',
-      STRING_VALUE_STAT.SS
  GOTO 99
ELSE IF (IOSTATUS.NE.HGSC_VALID) THEN
  PRINT*, 'Return status from the first WRITE ',
-      'of set "STRING_SET"=', IOSTATUS
  GOTO 99
ELSE
  PRINT*, 'First WRITE of set "STRING_SET" was successful'
  PRINT*, ' '
ENDIF

C      Reading REAL_SET from LCN
EXEC DDA READ SET = 'REAL_SET', STATUS = IOSTATUS;
IF (IOSTATUS .EQ. HGSC_CHECK_STATUS) THEN
C      Checking each individual value status to see which one(s)
C      had the error.
C
  IF ((REAL1_VALUE_STAT.VS .NE. HGSC_VALID) .AND.
-      (REAL1_VALUE_STAT.VS .NE. HGSC_NAN)) THEN
    PRINT *, 'REAL1_VALUE read bad data, status=',
-      REAL1_VALUE_STAT.VS
  ENDIF
  IF ((REAL2_VALUE_STAT.VS .NE. HGSC_VALID) .AND.
-      (REAL2_VALUE_STAT.VS .NE. HGSC_NAN)) THEN
    PRINT *, 'REAL2_VALUE read bad data, status=,'
```

```

-          REAL2_VALUE_STAT.VS
      ENDIF
      GOTO 99
    ELSE IF (IOSTATUS.NE.HGSC_VALID) THEN
      PRINT*, 'Return status from the READ ',
-        'of set "REAL_SET"=', IOSTATUS
      GOTO 99
    ELSE
      PRINT*, 'READ of set "REAL_SET" was successful'
      PRINT*, ' '
    ENDIF

C      if points are NAN, set to some real value
      IF (REAL1_VALUE_STAT.VS .EQ. HGSC_NAN) REAL1_VALUE=10.0
      IF (REAL2_VALUE_STAT.VS .EQ. HGSC_NAN) REAL2_VALUE=20.0

      PRINT*, 'REAL1_VALUE =', REAL1_VALUE
      PRINT*, 'REAL2_VALUE =', REAL2_VALUE

      REAL1_VALUE = REAL1_VALUE + 11.1
      REAL2_VALUE = REAL2_VALUE + 22.2

      PRINT*, ' '
      PRINT*, 'New values being written'
      PRINT*, 'REAL1_VALUE =', REAL1_VALUE
      PRINT*, 'REAL2_VALUE =', REAL2_VALUE
      PRINT*, ' '

C      Writing REAL_SET to LCN
      EXEC DDA WRITE SET = 'REAL_SET', STATUS = IOSTATUS;
      IF (IOSTATUS .EQ. HGSC_CHECK_STATUS) THEN
C        Checking each individual value status to see which one(s)
C        had the error.
C
        IF (REAL1_VALUE_STAT.SS .NE. HGSC_VALID) THEN
          PRINT *, 'Write of REAL1_VALUE bad, status=',
-            REAL1_VALUE_STAT.SS
        ENDIF
        IF (REAL2_VALUE_STAT.SS .NE. HGSC_VALID) THEN
          PRINT *, 'Write of REAL2_VALUE bad, status=',
-            REAL2_VALUE_STAT.SS
        ENDIF
        GOTO 99
      ELSE IF (IOSTATUS.NE.HGSC_VALID) THEN
        PRINT*, 'Return status from the WRITE ',
-        'of set "REAL_SET"=', IOSTATUS
        GOTO 99
      ELSE
        PRINT*, 'WRITE of set "REAL_SET" was successful'
        PRINT*, ' '
      ENDIF

      STRING_VALUE = 'OpenDDA Demo Ending'

```

```
C      Writing STRING_VALUE to LCN
      EXEC DDA WRITE SET = 'STRING_SET', STATUS = IOSTATUS;
      IF (IOSTATUS .EQ. HGSC_CHECK_STATUS) THEN
C      Since only one point exists in this set, we know the
variable
C      that had the error.
C
      PRINT *, 'Second write of STRING_VALUE bad, status=',
-      STRING_VALUE_STAT.SS
      GOTO 99
      ELSE IF (IOSTATUS.NE.HGSC_VALID) THEN
      PRINT*, 'Return status from the second WRITE ',
-      'of set "STRING_SET"=', IOSTATUS
      GOTO 99
      ELSE
      PRINT*, 'Second WRITE of set "STRING_SET" was successful'
      PRINT*, ' '
      ENDIF

99      CONTINUE
C      Terminating Application Environment
      EXEC DDA TERMINATE STATUS = IOSTATUS;
      IF (IOSTATUS .NE. HGSC_VALID) THEN
      PRINT *, 'ERROR DURING TERMINATE, STATUS=', IOSTATUS
      GOTO 999
      ENDIF

999      CONTINUE
      END
```

ddademo.def listing

Refer to Section 7—OpenDDA Reference Manual

The following is a listing of the `ddademo.def` file.

```
DATA_PRIORITY=NOCONTROL;
INITIAL_VALUES
  STORE_CODE=STORE;
CIO_SIGNATURE=ERROR;
INCONSISTENCY=ABORT;
PRINT
  VALUES=ON
  STATS=ON
  MESSAGES=ON;
```

12. OpenDDA Development Guidelines and Standards

12.1 Overview

Purpose

The purpose of this section is to provide supplemental information on the *AIA Development Guidelines and Standards* document when developing FORTRAN or C/C++ applications using Open Data Definition and Access (OpenDDA).

Prerequisite

To understand the system framework of the future Honeywell IAC products, please refer to the *AIA Development Guidelines and Standards* document. It provides a functional description of the framework, known as the Application Integration Architecture (AIA).

AIA framework

The AIA framework is defined as a set of interfaces built on lower-level interfaces, with products supporting the implementation of those interfaces.

The following is a hierarchical view of these layers:

Applications
Application Enablers
System Enablers
Standard System Services
Operating System
Hardware

Scope

The scope of this section includes information on FORTRAN and C/C++ applications as well as OpenDDA as an Application Enabler.

Honeywell SDK's

This section is in addition to the Honeywell OpenUSE program's Software Development Kits (SDKs). Other documents in the OpenUSE SDKs are listed below.

References

The following is a list of other development guideline and standard documents you may want to reference:

Publication Title	Publication Date	Binder
PDA AIA Development Guidelines and Standards	March 1994	PD12-100
Honeywell IAC Human Interface Style Guide	March 1994	SDK1
PDA Implementation Guide	March 1994	SDK1

12.2 Applications

Definition

Applications are software entities that directly execute the customer's mission. They may be provided by Honeywell IAC, customers, or third- parties. All other layers within the AIA structure exist only to support applications.

12.3 Application Enablers

Definition

Application Enablers (AEs) are software facilities that solve common problems for large sets of applications in a consistent way.

Application Enablers are less oriented toward operating system services and more toward common useful products.

Function

An Application Enabler might consist of a set of functions common to a class of applications and may be implemented once, and made available in a library, rather than being reimplemented in each application.

An AE may also be a support tool to

- create applications,
- check applications for conformance,
- help to create test suites, and
- enable data storage in standard formats for use by common applications.

OpenDDA product description

OpenDDA is a platform independent software layer providing a standard application programming interface and assists in developing and implementing applications with the following qualities:

- perform efficiently
- read and write varying amounts of heterogeneous data
- access one or more external data sources
- are able to migrate and
- are portable.

Platform

OpenDDA executes on the Application Modulex. This platform provides an open environment for the development and execution of custom control applications created by the customer.

12.4 FORTRAN 77 Standards

Standard FORTRAN 77

To ensure portability and reliability in your FORTRAN applications, your software should comply to the Federal Information Processing Institute Standard FIPS 69-1. This is equivalent to the ANSI Standard and International Standards Organization ISO 1539-1980.

HP-UX FORTRAN 90 compiler

The HP-UX FORTRAN 90 compiler available on Honeywell's Application Modulex is a superset of FORTRAN 77. It is compliant with both the International ISO standard (ISO/IEC 1539:1991) as well as the ANSI standards.

12.5 FORTRAN Extensions and Programming Techniques

Acceptable extensions

There are several FORTRAN 77 extensions accepted by most compilers. Two of the most common and useful are

- variable names greater than six characters, and
- the ability to use structures within your application.

Both are supported by the HP-UX FORTRAN 90 compiler and are acceptable to use and still maintain portable FORTRAN code.

Argument matching

Many older FORTRAN implementations allow you to omit some arguments when you are calling a subroutine.

Avoid this extension and always use ALL arguments when calling a subroutine.

Recursion

Many compilers claim to allow recursion, but few implement it correctly. Do not assume that recursion works without some careful experimentation.

Along with recursion, do not depend on the `STATIC` or `AUTOMATIC` keywords in the FORTRAN language.

Passing different size data types

Most UNIX hardware platforms follow the IEEE 754 floating-point standard. With this floating-point representation, `REAL` numbers are not a prefix for `DOUBLE PRECISION` numbers.

- If a subroutine has `REAL` arguments, you must pass `REAL` data to it.
- If a subroutine has `DOUBLE PRECISION` arguments, you must pass `DOUBLE PRECISION` data to it.

IMPLICIT NONE

Including the `IMPLICIT NONE` statement in your FORTRAN applications is highly recommended. This statement forces the declaration of all variables, instead of relying on the FORTRAN variable defaults of names beginning with the letters “I” through “N” as type `INTEGER` and all others as type `REAL`.

Array subscripts

Many FORTRAN compilers do not allow `REAL` numbers to be used as array subscripts.

Array subscripts must be `INTEGER` (or `INTEGER*2`) quantities.

12.6 Information on C/C++

Reference for C/ C++ information

Refer to the *PDA AIA Development Guidelines and Standards* in the *PD12-100* binder for information about C/C++.

13. OpenDDA Compile and Link Scripts

13.1 C Compile and Link Script File

Filename

This file is `/opt/DDA/examples/dda_create_c_executable`.

Script listing

```
# Example script that creates an executable for an OpenDDA application
# written in C using an ANSI C compiler. The application name with no
# extension is entered as a parameter.
#
# The options used are as follows:
#
# -Z    Allows dereferencing of null pointers.
# -Aa   Compile under ANSI mode.
#
cc -Z -Aa -I/opt/DDA/include -c $1_src.c
cc -Aa -I/opt/DDA/include -o $1 $1_src.o \
    -L/opt/DDA/lib -ldda -lddagdx -lddagen \
    -L/opt/TDC_Open/common/lib -llxs -lhiber
exit
```

Debug mode script

There is also an example script to build the application in debug mode. The debug script file is:
`/opt/DDA/examples/dda_create_c_debug_executable`.

Makefile

There is also a sample Makefile to create a C application. The Makefile is:
`/opt/DDA/examples/dda_c_examples.make`.

13.2 C++ Compile and Link Script File

Filename

This file is `/opt/DDA/examples/dda_create_cpp_executable`.

Script listing

```
# Example script that creates an executable for an OpenDDA application
# written in C using an ANSI C++ compiler. The application name with no
# extension is entered as a parameter.
#
# The options used are as follows for the HP C++ compiler version
# A.03.50:
#
# -Z Allows dereferencing of null pointers.
# -Aa Request ANSI mode HP C++ preprocessor.
# +a1 Causes the translator/compiler to produce ANSI C style
# declarations.
# +O0 Suppress optimizations.
#
CC -Z -Aa +a1 +O0 -I/opt/DDA/include -c $1_src.c
CC -Aa +O0 -I/opt/DDA/include -o $1 $1_src.o \
-L/opt/DDA/lib -ldda -lddagdx -lddagen \
-L/opt/TDC_Open/common/lib -llxs -lhiber
exit
```

Debug mode script

There is also an example script to build the application in debug mode. The debug script file is:
`/opt/DDA/examples/dda_create_cpp_debug_executable`.

13.3 FORTRAN Compile and Link Script File

Filename

This file is `/opt/DDA/examples/dda_create_fortran_executable`.

Script listing

```
# Example script that creates an executable of an OpenDDA application.
# The application name with no extension is entered as a parameter.
#
# The options used are as follows:
#
# -w Suppress warning messages.
# -C Enables range checking.
# -u Forces the types of identifiers to be implicitly UNdeclared.
# +es Allows more than 72 characters on a single source line.
#
fort77 -wCu +es -I/opt/DDA/include -o $1 $1_src.f \
-L/opt/DDA/lib -ldda -lddagdx -lddagen \
-L/opt/TDC_Open/common/lib -llxs -lhiber
exit
```

Debug mode script

There is also an example script to build the application in debug mode. The debug script file is:
`/opt/DDA/examples/dda_create_fortran_debug_executable`.

Makefile

There is also a sample Makefile to create a FORTRAN application. The Makefile is:
`/opt/DDA/examples/dda_fortran_examples.make`.

14. Glossary

Alias Variable

A variable that your application loads with an ASCII representation of an External Reference

Anchored Access

Anchored Access is the ability to specify the starting index of an External Reference array and read or write successive values into a HLL variable array.

Ancillary Variable

A general term used to refer to all EDB Data Access variables other than the Value Variable.

Application Module^x (A^xM)

A module on the Local Control Network that processes data points to provide supervisory control. Standard algorithms and custom programs written in Control Language are used to process data points.

Cache

Whenever OpenDDA accesses a DCS to resolve an External Reference, the information returned (Internal Id, data type, etc.) is entered into a Cache located in the HP-UX. Subsequent resolutions for this External Reference are satisfied by the Cache instead of accessing the DCS, which greatly improves performance.

Cache Management Tool (CMT)

The Cache Management Tool allows examination and/or deletion of Cache entries. It also supports checkpointing the cache.

CL

CL (Control Language) is a Honeywell proprietary software language used to build programs which will execute in an Application Module (AM).

Custom Display, (Schematic)

See Schematic.

Data Access Set

A Data Access Set is a collection of EDB statements whose Value Variables and Ancillary Variables will be acted on in one operation such as an EXEC DDA READ or EXEC DDA WRITE statement.

Data point

A TDC 3000^x data point is a named collection of related parameters and processing involved with the acquisition or calculation of a main item of information and may optionally include the data for an associated control loop.

A data point has three parts: a tagname (also called point name or point ID), parameters, and functions.

DCS

Distributed Control System (e.g., TDC 3000^x).

Depopulate HLL

HLL variables associated with the External Data Statement will be moved to OpenDDA in preparation for writing to an external location.

Dynamic Selection Set

A Dynamic Selection Set is a collection of EDB statements whose External References will be modified in one operation by an EXEC DDA MODIFY CIO statement.

Dynamic Selection Set Functionality (Dynamic Data Referencing)

Dynamic Selection Set Functionality (also known as Dynamic Data Referencing) allows an OpenDDA Application to modify the External Data References within the memory resident CIO to refer to different External Data values.

EDB (External Data Block)

An External Data Block establishes a relationship between your HLL variables and the external data values.

EXEC DDA Statement

EXEC DDA Statements are syntax that you add to your OpenDDA application source code to perform OpenDDA functions at run time.

External Data

A general term that refers to data values read or written from a DCS.

External Data Reference

The identification of a data value on a DCS. It consists of:

- a name, which identifies the External Reference - for example, point1.paramter6(2)
- a status, which indicates whether or not the External Reference represents a valid external data item

External Reference

Shorthand notation for External Data Reference.

External Reference Component

External Reference Names are composed of fields called "components". The name has the form POINT.PARAMETER(INDEX). The components are:

- the POINT, the part before the period
- the PARAMETER, the part after the period and before the optional index
- the INDEX, the optional part after the PARAMETER and inclosed in parenthesis

External Reference Name

The externally known ASCII identification of a data value on a DCS. For example, point1.parameter6(2).

GDX (Generic Data eXchange)

GDX is a Honeywell set of software providing the exchange of data between the Host platform (HP-UX) and the DCS (TDC3000^x).

General Output

The default output destination as defined by the Host Platform.

General Output file

A file OpenDDA creates to contain the general output of an executed application. This includes any normal printing statements of an application, along with any other application output.

HLL - High Level Language

Computer languages (such as Fortran or C/C++) used to write OpenDDA applications.

Host Platform

The Host Platform is a hardware and software computer system that serves as an application development environment for OpenDDA. Host Platforms supported by OpenDDA include

- Honeywell's Application Module^x (A^xM), which includes Hewlett-Packard's PA-RISC coprocessor running HP-UX operating system version 9.05 or later
- A Hewlett-Packard workstation based on Hewlett-Packard's PA-RISC coprocessor running HP-UX operating system version 9.05 or later.

Inconsistency Error

An inconsistency error occurs when OpenDDA detects a mismatch in data size or array size between the application variable and the value being moved into or out of the application. An example is when the definition of an external data item in the CIO file does not match the corresponding HLL variable definition in data size and array size.

Input/Output

You can specify/control when I/O occurs to the data platform. HLL variables are populated/depoppedulated by OpenDDA-generated code within the body of the application.

Install Application

OpenDDA command line function which prepares an OpenDDA Application for on-line execution.

Internal Id

DCS specific internal representation for an External Reference Name. Typically, accessing data using the Internal Id is much faster than using the equivalent External Reference Name.

Local Control Network (LCN)

The local area network that interconnects the nodes in a TDC 3000^x system.

Network Gateway

The node that logically connects two LCNs together. The Network Gateway name (also referred to as the LCN name) is prefixed to the point name when used in an External Reference Name. An example is: SS\POINT1.PARAMETER5 where "SS" is the LCN (or Network Gateway) name. The LCN name can also be used for local LCN access when there is no Network Gateway as long as the local LCN is configured with a name.

Off-line Application

OpenDDA Application that is not logically connected to the DCS for execution. These applications cannot be started by the DCS and cannot receive events from the DCS.

On-line Application

OpenDDA Application that has been logically connected by use of the Install Application command line function to a DCS for execution. These applications are typically started by the DCS and may receive events from the DCS.

OpenDDA

Open Data Definition and Access.

A platform independent data abstraction layer that

- provides a standard application programming interface
- assistance for developing and implementing applications with the following qualities:
 - perform efficiently
 - read and write varying amounts of heterogeneous data
 - access one or more external data sources.

OpenDDA Application

A Program written in FORTRAN or C/C++ that uses OpenDDA extensions to the language (EDB and EXEC DDA statements) to access data on a DCS using DCS independent syntax.

OpenDDA Application Directory

The directory where your OpenDDA application physically resides. This directory is usually (but not required to be) where the application was created, compiled, and linked. The files which support your application (i.e. CIO file, DEF file) are also contained in this directory. For on-line applications, this directory is the directory from which the Install Application was performed. For off-line applications, this directory is the current directory.

OpenDDA Status Mnemonics

Mnemonic names for constants used in an OpenDDA application when dealing with OpenDDA status values. The mnemonics are used exclusively throughout the OpenDDA user documentation whenever a reference is made to an OpenDDA status value.

OpenDDA Status Values

Values returned to an application by an OpenDDA Execution Statement. These values are used to indicate various degrees of success or failure.

Parameters(TDC) (also point.parameters)

A TDC 3000^x database parameter consists of a single value. The values can be

- input data_flow, temperature, pressure, ...
- tuning parameters_gain, time constants, bias, recipe data, ...
- output data_valve outputs, alarm states, ...

The TDC 3000^x bookset includes parameter reference dictionaries which list and define every parameter associated with specified system devices.

Persistence of External References

The procedure of modifying the External Reference Components, saving the memory resident CIO to disk, reloading the CIO later, and using the External References, is known as "persistence of External References". The dynamically selectable External References persist over time by writing them to the CIO on disk.

Platform

The term platform is used to reference the hardware and software computer system serving as an external data source. Platforms supported by OpenDDA include

- Honeywell's Application Module^x (A^xM)

Point name (also called data point name, tagname)

A point name is a unique name identifying a data point.

Each name is from one to sixteen characters long and the first character must always be an alphanumeric character.

Examples: TIC100, FC_2000, PUMP200A

Populate HLL

HLL variables associated with the External Data Statement will be populated by OpenDDA with values read from the external locations.

Remap CIO

Remap CIO is the functionality which allows an OpenDDA application to dynamically associate the HLL variables to a different group of external data by reading multiple CIO files into memory.

Remote Node

A node on a network other than the A^XM node that is used to obtain access to external data.

Schematic (Custom Display)

User-defined graphic and textual representations of the process or any other available information for the console operator on the TDC 3000^x system.

TDC

Total Distributed Control - the Honeywell design philosophy of full distribution of architecture, full integration of functions, and distributed risk, allowing for continuous expansion and capability.

Index

-
- .cio file, 78, 84
- .def file, 86
- .login file, 34, 41
- .set file, 78, 81, 84
- .set_idx file, 78, 82, 84
- .src.f file, 82, 84
- A**
 - Application Enabler, 164
 - AIA framework, 161
 - AIA guidelines, 161
 - Alias Variable, 119
 - syntax, 119
 - Alias Variables
 - Loading, 121
 - ALL set, 116
 - Application state
 - online/offline
 - online/offline state, 24
 - AXM
 - move application from remote platform, 55
- B**
 - Build application, 48, 51
- C**
 - Cache
 - entries, 133
 - size, 132
 - Cache Maintenance Tool, 131
 - Commands, 135
 - Examples, 140
 - Overview, 131
 - Purpose, 131
 - Cache size, 132
 - CB function (Complete Build), 84
 - cio
 - signature, 110
 - CIO signature
 - How EDBs can differ, 111
 - How EDBs cannot differ, 111
 - CL
 - program to execute OpenDDA application, 89
 - OpenDDA command
 - OpenDDA, 34
 - Command line
 - components, 66
 - error messages, 72
 - examples, 73
 - messages, 71
 - options, 70
 - qualifiers, 69
 - Compiler options
 - script files
 - compiler options
 - script files
 - compiler option, 39, 40
 - Complete Build, 84
 - Completion status, 74
 - Create new application, 46
- D**
 - Data types
 - passing different sizes, 166
 - DCS data, 27
 - DDA_NULL, 122
 - DDA_REDIRECT_OUTPUT, 41
 - DDAOPTS, 67
 - De-Install Application, 92
 - DS function (De-Install Application), 92
 - DV function (Validate Definition File Syntax), 86
 - Dynamic Selection
 - concept, 117
 - How it works, 117
 - ALL Dynamic Selection Set
 - ALL Selection Set, 116
 - Dynamic Selection Sets, 113
 - Comparison with Remap CIO, 113
 - Identifying dynamic components, 118
 - Introduction, 113
 - Overview, 113
 - terminology, 115
 - tools, 114
 - valid Location Specifications, 114
- E**
 - EE function (Expand EXEC DDA Statements), 81
 - Environment variables, 41
 - DDAOPTS, 67
 - Examples
 - cb function, 85
 - command line messages, 73
 - completion status, 74
 - ds function, 92
 - dv function, 86
 - ee function, 83
 - build application
 - build an application, 49, 52
 - create new application
 - create a new application, 47
 - execute offline application from AXM
 - execute an offline appl. from an AXM, 60
 - execute offline application from remote node
 - execute an offline appl. from a remote node, 62
 - execute online application from AXM

Index

- execute an online appl. from an AXM, 63
- install online application
 - install an online application, 58
- move remote application to AXM**
 - move a remote appl. to an AXM**, 56
- is function, 89
- OpenDDA files, 35
- pe function, 80
- EXEC statement
 - LOAD ALIAS, 129
 - LOAD MODIFY_CODE, 130
 - MODIFY CIO, 127
- Expand EXEC DDA Statements, 81
- External data
 - types, 29
- External Reference Keyword
 - POINT_NAME
 - PARAM_NAME
 - INDEX_NAME
 - FULL_NAME;, 119
- External Reference Status Variable, 123

F

- Facility Code, 99
- Filespec arguments, 69
- Argument matching
 - argument matching, 166
- Array subscripts
 - array subscripts, 166
- FORTRAN
 - extensions, 166
 - standards, 165
- IMPLICIT NONE statement
 - IMPLICIT NONE statement, 166
- Recursion
 - recursion;STATIC keyword, 166
- Cache entry freshness, 133
- FULL_NAME Keyword, using, 120

H

- H, 111
- Help, 75
- HHL, 28
- How to
 - build a DDA command line, 68
- HP-UX FORTRAN compiler, 165

I

- Arrays and Indexes, 125
- Install Application, 88
- IS function (Install Application), 88

M

- Message Number, 100
- mnemonic, status, 97

- Modify Code Variable, 124

O

- o option, 79
- Offline application, 24, 25
 - execute application from AXM, 59
- Offline directory
 - definition
 - offline, 25
- Online application
 - definition, 26
 - execute application from AXM, 62
 - install, 57
- Online directory
 - definition
 - online, 26
- Online help, 75
- File protection
 - file protection
 - file protection, 43
- OpenDDA
 - file types;File extensions;File types;.f file;.def file;.set file;.set_idx file;.cio file, 94
 - application
 - source file, 29
 - types, 28
 - build application, 48
 - command line components, 66
 - command line options, 70
 - command line qualifiers, 69
 - compiler options
 - suppress warning messages
 - suppress warning messages;Suppress warning messages
 - warning messages
 - compiler options, 39
 - track compiler errors
 - suppress warning messages;Compiler errors, 39
 - completion status, 74
 - create new application, 46
 - data access, 27
 - definition, 24
 - environment variables, 41
 - error messages, 72
 - execute offline application from AXM, 59
 - execute offline application from remote node, 61
 - execute online application from AXM, 62
 - features, 23
 - file examples, 35
 - install application online, 57
 - make files, 36
 - messages, 71
 - online help, 75
 - Host Platform;Host platforms
 - valid OpenDDA, 21, 175
 - reference information, 18
 - requirements
 - user, 22
 - separate EDB, 51

Index

Template files
 template files, 35
Optional parameter indicators, 19

P

Status parameters;RET_STAT;DET_STAT
 status, 91
Parse External Data Block, 78
PE function (Parse External Data Block), 78
Problems
 (see Troubleshooting OpenDDA), 18

R

READONLY, 42
READWRIT, 42
References
 application development, 162
Remapping CIO Files, 107
 Functionality, 109
 Overview, 107
 Similarity to Equipment List, 110
 tools, 109
Remote platform
 execute offline application from, 61
 moving application to AXM, 55
 restrictions
 remote applications;, 79
RW_LCN_I, 42

S

Make files, 36
Script files, 76
Security, 42
Security switch, 42

OpenDDA Sets
 OpenDDA, 116
Sets
 Data Access Sets
 Dynamic Selection Sets, 116
Severity, status, 100
Shared libraries, 38
CIO signature, 110
Standard Output
 redirecting, 72
Status Mnemonic, 97
Status Values, 97
Status Values, interpreting, 99
Status, determining corrective action, 104

T

Troubleshooting OpenDDA, 18

U

Upper/lowercase, 19
 CL program case-sensitivity
 case-sensitivity, 90
 command line options, 70
 command line qualifiers, 69
 OpenDDA file extensions, 94
User requirements, 22

V

Validate Definition File Syntax, 86

X

-X option, 79

FAX Transmittal

FAX No.: (602) 313-4842

TO: _____

Total FAX pages: _____
(including this page)

Reader Comments

Title of Document: **OpenDDA User's Guide**

Document Number: **DD11-300**

Issue Date: **12/98**

Comments: _____

Recommendations: _____

FROM:

Name: _____ **Date:** _____

Title: _____

Company: _____

Address: _____

City: _____ **State:** _____ **ZIP:** _____

Telephone: _____ **FAX:** _____

FOR ADDITIONAL ASSISTANCE:

Write	Call
Honeywell Inc. Industrial Automation and Control Automation College 2820 West Kelton Lane Phoenix, AZ 85053-3028	Technical Assistance Center (TAC) 1-800-822-7673 (48 contiguous states except Arizona) 602-313-5558 (Arizona)

Honeywell

Industrial Automation and Control

Automation College

2820 W. Kelton Lane

Phoenix, AZ 85053-3028